

Operating Systems & Concurrency

A Comprehensive Reference for SDE Interviews & Online Assessments

Compiled 12 July 2026

Contents

1	Introduction and scope	4
1.1	How this document is organized	4
1.2	Conventions	5
2	Operating-system fundamentals and structure	5
2.1	What an operating system is	5
2.2	User mode and kernel mode	6
2.3	System calls	6
2.4	Interrupts, traps, exceptions, and faults	8
2.5	Operating-system structure	8
2.6	The boot process	9
2.7	Where services live	10
3	Processes and inter-process communication	10
3.1	From program to process	10
3.2	The process address space	11
3.3	Process states	11
3.4	The process control block (PCB)	12
3.5	Context switches	12
3.6	Creating processes: <code>fork</code> , <code>exec</code> , <code>wait</code>	13
3.7	Copy-on-write	13
3.8	Zombies and orphans	14
3.9	Inter-process communication	14
3.9.1	Pipes	15
3.9.2	Shared memory	15
3.9.3	Message queues	15
3.9.4	Signals	15
3.9.5	Sockets	15
3.9.6	Choosing a mechanism	16
3.9.7	Producer–consumer through IPC	16
4	Threads and concurrency models	17
4.1	Threads versus processes	17
4.2	Why threads, and their hazards	17
4.3	User-level versus kernel-level threads	18

4.4	Multithreading models	18
4.5	Concurrency versus parallelism	19
4.6	Thread pools	19
4.7	Thread safety, reentrancy, and thread-local storage	19
4.8	Lightweight threads: fibers, green threads, coroutines	20
5	CPU scheduling	20
5.1	Scheduling fundamentals	20
5.1.1	Timing attributes	20
5.1.2	The algorithms at a glance	21
5.2	First Come First Serve (FCFS)	22
5.3	Shortest Job First (SJF) — non-preemptive	22
5.4	Shortest Remaining Time First (SRTF) — preemptive SJF	23
5.5	Round Robin (RR)	24
5.6	Priority scheduling (preemptive)	25
5.7	Priority scheduling with I/O bursts	26
5.8	Multilevel Queue (MLQ) scheduling	28
5.9	Multilevel Feedback Queue (MLFQ) scheduling	29
5.10	Assessment templates: average-turnaround drills	30
5.10.1	Template A — Preemptive SJF (SRTF), five processes	30
5.10.2	Template B — Preemptive priority, five processes	31
6	Process synchronization	32
6.1	Independent and cooperative processes	32
6.2	The race condition	32
6.3	The producer–consumer (bounded-buffer) problem	33
6.4	The printer–spooler problem	34
6.5	The critical-section problem	35
6.6	Software and hardware solutions	35
6.6.1	Lock variable	35
6.6.2	Test-and-set instruction	36
6.6.3	Turn variable (strict alternation)	36
6.7	Semaphores	37
6.7.1	Counting semaphore	37
6.7.2	Binary semaphore	37
6.7.3	A worked question: maximum processes in the critical section	37
6.8	Classic synchronization problems	38
6.8.1	Producer–consumer with semaphores	38
6.8.2	Readers–writers	38
6.8.3	Dining philosophers	39
6.9	The concurrency toolkit in practice	40
7	Deadlocks	41
7.1	What a deadlock is	41
7.2	Resource-allocation graphs	42
7.2.1	Single-instance resources	42

7.2.2	Multiple-instance resources	42
7.3	Handling deadlocks	42
7.4	Deadlock prevention	42
7.5	Deadlock avoidance: the Banker's algorithm	43
7.5.1	Checking whether a given ordering is safe	44
7.6	Detection and recovery	44
7.7	How many resources guarantee no deadlock	45
8	Memory management	45
8.1	The goal and the memory hierarchy	45
8.2	Fixed (static) partitioning	46
8.3	Variable (dynamic) partitioning	46
8.3.1	Placement algorithms	46
8.4	Paging	47
8.4.1	Pages, frames, and the page table	47
8.4.2	Address translation	47
8.4.3	Page-table entry format	48
8.4.4	Multilevel paging	48
8.4.5	Inverted page table	48
8.5	Segmentation	48
9	Virtual memory	49
9.1	Demand paging	49
9.1.1	Effective memory access time	50
9.2	The translation lookaside buffer	50
9.3	Page replacement	50
9.3.1	FIFO and Belady's anomaly	50
9.3.2	Optimal	51
9.3.3	Least recently used	51
9.3.4	Most recently used	51
9.4	Thrashing	52
9.5	Related mechanisms	52
10	File systems	52
10.1	Files, attributes, and operations	52
10.2	File allocation methods	53
10.2.1	Contiguous allocation	53
10.2.2	Linked allocation	53
10.2.3	Indexed allocation	53
10.3	The Unix inode	53
10.4	Free-space management	54
11	I/O systems and disk scheduling	54
11.1	Disk architecture	54
11.2	Disk access time	54
11.3	Disk scheduling algorithms	55
11.4	How the CPU talks to devices: polling, interrupts, and DMA	56

11.4.1 Which to use — and interrupt livelock	56
11.4.2 Bounded queues and tail latency	57
12 Protection and security	57
12.1 Goals and principles	57
12.2 Domains and the access matrix	58
12.3 Security violations	58
12.4 Threats	58
12.5 Defenses	59
13 Appendix A — Programming paradigms	59
13.1 Monolithic	59
13.2 Procedural	59
13.3 Modular	60
13.4 Object-oriented	60
14 Appendix B — Rapid-fire interview questions	61

1 Introduction and scope

This document is a single, self-contained reference for the operating-systems knowledge a software-engineering candidate needs for interviews and online assessments at companies such as Uber, Microsoft, Amazon, Google, and Cisco. It is built to be studied end to end without consulting any outside resource.

The material is drawn from two sources and fused into one continuous treatment:

- **A full lecture course**, captured as worked whiteboard sessions — CPU scheduling, process synchronization, deadlocks, memory management, virtual memory, file systems, disk and I/O, and protection. Every numeric example, Gantt chart, resource-allocation matrix, formula, code fragment, and diagram from those sessions is reproduced and independently re-derived.
- **Standard interview and assessment topics** the lectures do not cover — the user/kernel boundary and system calls, process creation and IPC, threading models, and the code-level concurrency toolkit (mutexes, spinlocks, atomics, memory ordering) — researched and written to the same standard so the reference is complete.

Concepts that appear on real Cisco intern online assessments are called out where they arise and are worked on the exact templates those assessments use, so practice maps directly onto the test.

1.1 How this document is organized

The sequence follows the natural dependency order of the subject: it begins with what an operating system is and how a program talks to it, builds up through processes, threads, and scheduling, then treats the three hard cores of the subject — synchronization, deadlocks, and memory — before finishing with virtual memory, file systems, I/O, and protection. Later sections assume the vocabulary established earlier.

1.2 Conventions

Recurring notes are set off as callouts:

Definition. A precise statement of a term.

Key point. An idea worth remembering, or the reason something works the way it does.

Trap. A common mistake, edge case, or misconception — the kind that costs marks on an assessment or credibility in an interview.

Interview. A question phrased the way an interviewer would ask it, with a crisp answer.

Formulae are typeset exactly; code is reproduced verbatim in fenced blocks; scheduling schedules, Gantt charts, and resource matrices are given as tables and have each been re-derived from first principles and checked against the worked answer. Where a numeric convention is arbitrary (for instance, whether a larger priority number means higher or lower priority), it is stated explicitly for each problem, because assessments deliberately vary it.

2 Operating-system fundamentals and structure

2.1 What an operating system is

An **operating system (OS)** is the software that sits between hardware and applications and does two jobs at once. As a **resource manager** it decides how the CPU, memory, disk, and network are shared among competing programs, fairly and safely. As an **abstraction layer** it hides the messy particulars of hardware behind clean, uniform interfaces, so that a program writes to a file without knowing whether the disk is spinning rust or flash, and opens a socket without driving the network card directly.

Definition — Kernel. The core of the operating system: the part that is always resident in memory and runs with full hardware privilege. It performs scheduling, memory management, and hardware communication. The *OS* is the larger package — kernel plus system libraries, shells, and utilities; the *kernel* is only its privileged heart.

Trap. A shell (`bash`), a compiler, or `ls` is **not** part of the kernel. These are ordinary user-space programs that ask the kernel for services through system calls. Likewise the C library (`libc`) is user-space; it *wraps* kernel calls, it is not inside the kernel.

2.2 User mode and kernel mode

If any program could execute any instruction and touch any memory, one buggy or malicious process could halt the machine or read another's data. To prevent this, the CPU supports **dual-mode operation**, governed by a hardware **mode bit**.

- **User mode** — the unprivileged mode ordinary applications run in. Code cannot execute privileged instructions or reach memory outside what the OS has granted it.
- **Kernel mode** (also *supervisor* or *ring 0*) — the privileged mode. Code may execute any instruction and access any address.

Privileged instructions — disabling interrupts, halting the CPU, loading the page-table base register, issuing raw I/O — are legal only in kernel mode. If a user-mode program attempts one, the hardware traps to the OS, which typically kills the offending process.

Key point. A **mode switch** is not a **context switch**. Crossing the user/kernel boundary (to service a system call, say) is a mode switch *within the same process*. A context switch changes which process is running; it involves mode switches but also saves and restores whole process state, and is far more expensive.

	User mode	Kernel mode
Privilege	Restricted	Unrestricted
Direct hardware access	No	Yes
A fatal bug here	Crashes one process	Crashes the system (kernel panic / BSOD)
Mode bit (typical)	1	0

2.3 System calls

A user program cannot simply *call* a kernel function the way it calls its own — that would let arbitrary code run with full privilege. Instead it enters the kernel through exactly one guarded gate: a **system call**.

Definition — System call. The mechanism by which a user-space program requests a service from the kernel, crossing from user mode to kernel mode through a controlled trap.

The mechanism, step by step:

1. The program places the arguments in registers and loads a **system-call number** into a designated register (for example `rax` on x86-64).
2. It executes a special instruction (`syscall` , `sysenter` , or the older `int 0x80`), which raises a **software interrupt (trap)**.
3. The hardware saves minimal state, flips the mode bit to kernel mode, and jumps to a fixed kernel entry point.
4. The kernel uses the system-call number to index the **system-call table** and dispatch to the right routine (e.g. `sys_read`).
5. The routine runs, the return value is placed in a register, the mode bit flips back, and an interrupt-return resumes the user program at the next instruction.

Because arguments travel in registers rather than on the stack (the user and kernel stacks are separate), and because large data is passed by a pointer the kernel must validate, a system call is much costlier than a plain function call — it flushes pipelines and pollutes caches.

	Library call	System call
Runs in	User mode	Kernel mode (after a mode switch)
Cost	Cheap	Expensive (trap + mode switch)
Portability	High (e.g. the POSIX C library)	Tied to the specific kernel
Examples	<code>printf</code> , <code>strlen</code> , <code>malloc</code>	<code>read</code> , <code>write</code> , <code>open</code> , <code>fork</code> , <code>brk</code>

Trap. A standard-library function is not automatically a system call. `printf` formats in user space and *then* calls the `write` system call; `malloc` manages a user-space heap and only drops into `brk` / `mmap` when it needs more pages. Know which is the wrapper and which is the `syscall`.

Interview. *Why can't a program call kernel functions directly?* Because a direct call does not change privilege level. The kernel must keep a hard boundary so user code can enter only through the trap handler, in a controlled way, with its arguments checked.

2.4 Interrupts, traps, exceptions, and faults

The CPU must react to events it did not plan for. These come in two flavours by *timing*: **asynchronous** (arriving independently of what the CPU is executing) and **synchronous** (produced by the current instruction).

- **Interrupt** — an *asynchronous* hardware signal (timer tick, disk-I/O completion, network packet, keystroke). The CPU finishes the current instruction, saves the program counter and status, looks up the handler in the **interrupt vector table**, and runs the **interrupt service routine (ISR)**. A *maskable* interrupt can be temporarily deferred; a *non-maskable* interrupt (e.g. an unrecoverable hardware error) cannot.
- **Trap** — a *synchronous, intentional* exception raised by an instruction, used to implement system calls and debugger breakpoints. Execution resumes at the following instruction.
- **Fault** — a *synchronous, unintentional* but often *recoverable* condition, such as a page fault or divide-by-zero. If the OS can fix it (e.g. by paging data in), execution retries the *faulting* instruction.
- **Abort** — an unrecoverable error.

Type	Source	Timing	Intentional?	Example
Interrupt	Hardware	Asynchronous	No	Timer, disk done, keystroke
Trap	Software	Synchronous	Yes	System call, breakpoint
Fault	Software	Synchronous	No	Page fault, divide by zero

Key point. A **page fault is not an error**. It is the normal signal that a referenced page is not currently in RAM; the OS fetches it from disk and retries. Demand paging is built on it.

Trap. An ISR generally must not sleep or block — it runs in a restricted context, often with interrupts disabled. It should do the minimum and defer heavy work (this deferral is exactly the idea behind the interrupt-then-poll designs discussed under I/O).

2.5 Operating-system structure

Kernels are organized along a spectrum trading **performance** against **fault isolation**.

- **Monolithic kernel** — scheduling, memory management, file systems, and drivers all run together in one kernel address space. Components talk by direct function call, so it is fast; but a bug anywhere (a bad driver) can crash the whole system. *Linux, classic Unix*.

- **Microkernel** — only the bare minimum (IPC, basic scheduling, basic memory) runs in kernel mode; drivers and file systems run as separate user-space server processes. A crashed driver can be restarted without taking down the OS, so it is reliable and modular — but every service interaction becomes IPC across the user/kernel boundary, which costs performance. *QNX, MINIX, L4, Mach.*
- **Hybrid kernel** — mostly monolithic for speed, with some subsystems structured as separate modules. *Windows NT; macOS (XNU) is commonly described this way.*
- **Modular / layered** — a monolithic kernel that loads drivers dynamically as **loadable kernel modules**. This is logical modularity, not a microkernel: once loaded, a Linux module runs with full kernel privilege.

	Monolithic	Microkernel
Kernel size	Large	Minimal
Speed	High (function calls)	Lower (IPC + mode switches)
Fault isolation	Weak (one bug crashes all)	Strong (a server can restart)
Example	Linux	QNX

Interview. *Why is a microkernel slower than a monolithic kernel?* In a monolithic kernel the file system reaches the disk driver by a direct function call. In a microkernel they are separate user processes, so the same interaction becomes message passing through the kernel — multiple mode switches and IPC copies per request.

Trap. Linux's loadable modules do **not** make it a microkernel. A loaded module executes in kernel space with full privilege; the isolation of a microkernel is exactly what it lacks.

2.6 The boot process

The path from power-on to a usable system is a hand-off through progressively more capable software:

1. **Power-on and POST.** The CPU begins executing firmware (BIOS or UEFI) from a fixed ROM address; the firmware runs a **power-on self-test** to check hardware.
2. **Firmware to bootloader.** The firmware locates a bootable device and loads the bootloader — BIOS from the disk's master boot record, UEFI from an `.efi` file on a system partition.
3. **Bootloader (e.g. GRUB).** It loads the **kernel image** and an initial RAM filesystem into memory and jumps to the kernel's entry point; it may offer a menu of kernels/OSes.
4. **Kernel initialization.** The kernel sets up memory management and paging, initializes the scheduler and devices, and mounts the root filesystem.

5. **First user process.** The kernel starts `init` / `systemd` as **PID 1**, the ancestor of all later processes.
6. **Services and login.** `init` starts daemons and networking and brings up the login prompt or graphical session.

Interview. *What is PID 1?* The first user-space process the kernel starts. It is the root of the process tree and adopts (and reaps) orphaned processes. *BIOS vs UEFI?* BIOS is the legacy 16-bit firmware that boots from an MBR (with a ~2 TB disk limit); UEFI is the modern replacement with GPT support, faster boot, a richer environment, and Secure Boot.

2.7 Where services live

Knowing which components run privileged and which do not explains many design and security questions.

- **Kernel space** — the process scheduler, memory manager, IPC primitives, file systems (and the virtual-filesystem layer), the network stack, and device drivers.
- **User space** — the windowing system, system libraries (`glibc` , `pthread`), shells, daemons (`sshd` , `cron` , `systemd`), and all applications.

Interview. *Why has a graphics-driver bug historically been able to blue-screen a machine?* Because drivers usually run in kernel space for speed, so a stray pointer there is a kernel fault, not a mere application crash. This is precisely the fragility microkernels trade performance to avoid.

3 Processes and inter-process communication

3.1 From program to process

A **program** is passive: an executable file on disk (an ELF binary, say) holding machine code and initial data. A **process** is that program *in execution* — an active entity with its own memory, registers, and operating-system bookkeeping. Running the same program twice creates two distinct processes. The process, not the program, is the unit to which the OS allocates resources.

Getting from source to a running process passes through a fixed toolchain:

HLL source $\xrightarrow{\text{compiler}}$ assembly / object files $\xrightarrow{\text{linker}}$ executable $\xrightarrow{\text{loader}}$ process in memory

The **compiler** translates high-level source into object files; the **linker** stitches object files and libraries into one executable, resolving cross-references; the **loader** (invoked by `exec`) maps that

executable into a fresh address space and starts it. An object/executable file is itself structured — a **header**, a **text** (code) section, a **data** section, a **BSS** section, a **symbol table**, **relocation information**, and **debugging information** — and the loader uses these to lay out the process image.

3.2 The process address space

When the loader builds a process, its memory is divided into regions with distinct purposes:

Region	Holds	Notes
Text (code)	Machine instructions	Read-only and shareable between processes running the same program
Data	Initialized globals/statics	
BSS	Uninitialized globals/statics	Zero-filled by the OS before <code>main</code>
Heap	Dynamic allocation (<code>malloc</code> / <code>new</code>)	Grows upward toward higher addresses
Memory-mapped region	Shared libraries, <code>mmap</code> ed files	Between heap and stack
Stack	Locals, arguments, return addresses	Grows downward toward lower addresses
Args/env	<code>argv</code> , environment strings	Highest addresses

Trap — stack vs heap. The **stack** is fast (allocation is just moving the stack pointer), automatically freed on return, but small and fixed-limit; a runaway recursion causes a **stack overflow**. The **heap** is large and flexible but slower, manually managed, and prone to fragmentation and leaks. The two grow toward each other; a collision is fatal.

3.3 Process states

A process moves through a lifecycle. The full **seven-state model** (the two “suspend” states appear once swapping to disk is considered):

- **New** — the PCB is being created; not yet admitted to memory.
- **Ready** — in memory, waiting for the CPU.
- **Running** — executing on a CPU.
- **Waiting / Blocked** — cannot proceed until an event occurs (I/O completion, a lock, a signal).
- **Terminated** — finished or killed; the PCB lingers until the parent reaps it.
- **Suspend-Ready** — was ready, but its memory has been swapped out to disk to relieve memory pressure; must be swapped back before running.

- **Suspend-Blocked** — was blocked *and* swapped out; when its event fires it becomes Suspend-Ready.

Key point. A thread has a simpler life — essentially **Running, Ready, Blocked** — because it is a unit of execution within a process and does not independently own memory to be swapped.

3.4 The process control block (PCB)

Definition — PCB. The kernel data structure that represents a process (in Linux, the `task_struct`). It lives in **kernel memory**, out of reach of user code.

A PCB records everything the OS needs to manage and resume the process:

- **Identification** — PID, parent PID, user/group IDs.
- **State** — its current lifecycle state.
- **Program counter** — the address of the next instruction.
- **CPU registers** — the saved register context when the process is not running.
- **Memory-management info** — page/segment tables, base/limit registers.
- **Accounting** — CPU time used, priority, limits.
- **I/O status** — allocated devices and the **open-file table**.

3.5 Context switches

Definition — Context switch. Saving the state of the running process into its PCB and loading another process's state, so the CPU can switch which process it runs.

The scheduler triggers a switch on a **timer interrupt** (a time slice expired — preemption), when a process **blocks** in a system call (e.g. `read`), or when an interrupt wakes a higher-priority process. What is saved and restored: the CPU registers (including PC and stack pointer) to/from the PCBs, plus pointing the MMU at the new process's page tables.

A context switch does **no useful application work** — **it is pure overhead**. Its cost is more than the visible register copying:

- **Direct** — cycles to save/restore registers, switch page tables, run the scheduler.
- **Indirect (the real cost)** — switching address spaces **flushes the TLB**, and the new process's data is cold in the CPU caches, so the first many memory accesses are slow.

Interview. *Why is a thread switch cheaper than a process switch?* Threads of one process share an address space, so a thread switch swaps only registers — the page tables are unchanged, so the **TLB is not flushed** and caches stay warm. A process switch changes the address space, paying the full TLB/cache penalty.

3.6 Creating processes: `fork`, `exec`, `wait`

Unix creates processes with a deliberately split, two-call design.

- **`fork()`** duplicates the calling process. It is called once but **returns twice**: in the parent it returns the child's PID, in the child it returns `0` (and `-1` on failure). The child receives an independent copy of the parent's address space and a *duplicate* of its open file descriptors.

```
pid_t pid = fork();
if (pid == 0) {    // child
    /* ... */
} else if (pid > 0) { // parent, pid = child's PID
    /* ... */
}
```

- **`exec()`** (the `execl` / `execvp` family) **replaces** the current process image with a new program loaded from disk. On success it **never returns** (there is nothing to return to); it keeps the same PID and, by default, the open file descriptors.
- **`wait()` / `waitpid()`** blocks the parent until a child terminates, delivering the child's exit status and letting the OS free the child's remaining resources (**reaping**).

Key point — why split `fork` and `exec`? The gap between them is where the child, still a copy of the parent, rewires itself: redirecting stdin/stdout, wiring up pipes, dropping privileges, or closing descriptors — *before* the new program takes over. This is exactly how a shell builds a pipeline such as `ls | grep foo`.

Trap — descriptor inheritance. Open descriptors survive `fork` **and** `exec` by default. A server that forks and execs a helper can leak its listening socket into the child. Mark sensitive descriptors `FD_CLOEXEC` (close-on-exec).

3.7 Copy-on-write

Copying a parent's entire address space on every `fork` would be wasteful — especially since the child often immediately `exec`s a different program. **Copy-on-write (COW)** avoids it.

On `fork`, the OS does not copy pages. It points the child's page table at the **same physical pages** as the parent, marks all of them **read-only**, and bumps each page's reference count. The processes share memory harmlessly as long as they only read. The instant either one **writes**, the MMU raises a protection **page fault**; the OS allocates a fresh frame, copies that one page into it, remaps the writer's page as read-write, and lets the write proceed. Only pages that are actually modified are ever copied.

Key point. COW makes `fork` cheap and is why the `fork -then- exec` idiom is efficient: almost nothing is copied before `exec` throws the address space away.

3.8 Zombies and orphans

Definition — Zombie. A process that has terminated but whose parent has not yet `wait`ed for it. The kernel must keep its exit status (a small PCB entry) until the parent collects it, so a zombie holds **no memory or CPU — only a PID**.

Definition — Orphan. A process whose parent terminated while it is still running. The OS **reparents** orphans to `init` / `systemd` (PID 1), which periodically `wait`s and reaps them.

Trap. You cannot `kill -9` a zombie — it is already dead; the signal has no one to act on. The cure is for the parent to `wait`, or to kill the *parent*, after which PID 1 adopts and reaps the zombie. A process that spawns many children and never reaps them (no `wait`, no `SIGCHLD` handler) leaks PIDs until the system cannot fork — a common daemon bug.

3.9 Inter-process communication

Because each process has a private address space, cooperating processes need explicit **IPC**. Everything reduces to two paradigms:

- **Message passing** — processes exchange messages through the kernel; the kernel moves the data and provides implicit synchronization. Simple and safe, but every message is copied `user→kernel→user`. (Pipes, message queues, sockets.)
- **Shared memory** — the kernel maps one physical region into several address spaces; processes then read/write it directly. **Fastest**, because after setup there is no kernel involvement and no copying — but the processes must synchronize access **themselves** (with semaphores/mutexes placed *inside* the shared region).

3.9.1 Pipes

A **pipe** is a one-way byte stream. An **anonymous pipe** (`pipe()`) yields a read end `fd[0]` and a write end `fd[1]` and is used between related processes (typically parent/child after `fork`). A **named pipe** / **FIFO** (`mknod`) is a special file in the filesystem, so *unrelated* processes can use it. Pipes have a finite kernel buffer: `write` blocks when it is full, `read` blocks when it is empty. Writing to a pipe whose read end is fully closed raises `SIGPIPE` (default: terminate).

3.9.2 Shared memory

The OS maps the same physical frames into multiple processes' page tables (`shm_open` + `mmap` , System V `shmget` / `shmat` , or anonymous `mmap`). Transfer is then pure user-space memory access — no syscalls, no copies. The kernel provides **no synchronization**, so a mutex or semaphore (kept in the shared region itself) is mandatory to avoid races.

3.9.3 Message queues

Kernel-maintained lists of discrete messages. Unlike a pipe's raw byte stream, a queue **preserves message boundaries** (one read returns exactly one message) and supports **types/priorities**, so a consumer can pull the highest-priority message first. Queues persist in the kernel until deleted.

3.9.4 Signals

A **signal** is an asynchronous notification — a software interrupt — that an event occurred. A process can catch it with a handler, ignore it, or take the default action. Common ones: `SIGINT` (Ctrl-C), `SIGKILL` (9, uncatchable), `SIGTERM` (15, catchable graceful stop), `SIGSEGV` (bad memory access), `SIGCHLD` (a child stopped/terminated), `SIGPIPE` (broken pipe).

Trap. Signals carry essentially no payload — just the number — so they are for *notification*, not data transfer. And handlers must be **async-signal-safe**: calling `malloc` or `printf` from a handler can corrupt state if the signal interrupted the same function mid-update.

3.9.5 Sockets

Sockets give bidirectional communication. **Unix-domain sockets** (`AF_UNIX`) connect processes on one machine through the kernel without touching the network stack (used by, e.g., a local database server). **Network sockets** (`AF_INET`) reach across machines: **TCP** is a reliable, ordered stream; **UDP** is unreliable, unordered datagrams.

3.9.6 Choosing a mechanism

Mechanism	Direction	Speed	Synchro- nization	Cross-ma- chine	Typical use
Anonymous pipe	One-way	Medium	Implicit (blocking)	No	<code>stdin / stdout</code> chaining (<code>ls grep</code>)
Named pipe (FIFO)	One-way	Medium	Implicit	No	Stream between unrelated processes
Shared memory	Two-way	Highest (zero-copy)	Manual (semaphores)	No	Large / high-frequency data
Message queue	Two-way	Medium	Implicit	No	Discrete, prioritized messages
Signals	One-way	Tiny payload	—	No	Async alerts, graceful shutdown
Unix socket	Two-way	Medium	Implicit	No	Local client-server
Network socket	Two-way	Slowest	Implicit	Yes	Distributed systems

3.9.7 Producer-consumer through IPC

The bounded-buffer producer-consumer problem (treated in full under synchronization) shows the paradigm trade-off concretely:

- **Shared memory + semaphores** — the buffer lives in shared memory guarded by a `mutex` plus an `empty` semaphore (initialized to the buffer size, counts free slots) and a `full` semaphore (initialized to 0, counts filled slots). Fastest, but every synchronization detail is the programmer's responsibility.
- **Message queue** — the producer `msgsnd`s and the consumer `msgrcv`s; the kernel blocks a producer on a full queue and a consumer on an empty one, so **no user-space synchronization is needed**. Simpler and robust, at the cost of kernel copying.

Interview. *Why is shared memory the fastest IPC?* After the one-time page mapping, the processes read and write RAM directly — no system calls and no user→kernel→user copying, unlike pipes, queues, and sockets. The price is that you must synchronize it yourself.

4 Threads and concurrency models

4.1 Threads versus processes

A **thread** is the basic unit of CPU execution within a process — a single sequence of instructions the scheduler can run independently. A process starts with one thread; giving it several lets those threads run concurrently while sharing the process's memory. The key to reasoning about threads is knowing exactly what they share and what they keep private:

Shared by all threads (process-wide)	Private to each thread
Address space: text, data, heap	Stack (locals, call frames)
Open files / sockets (descriptor table)	Registers, including the stack pointer
Global and static variables	Program counter
Signal handlers, accounting, PID	Thread ID, scheduling state, thread-local storage

The kernel tracks each thread with a **thread control block (TCB)** — TID, saved registers, PC, stack pointer, state, and a pointer back to the process's PCB.

Key point. Because threads share the heap and globals, they communicate simply by reading and writing shared variables — no IPC needed. That convenience is also the danger: shared writable state is exactly what creates race conditions, which is why synchronization (the next section) exists.

4.2 Why threads, and their hazards

Threads buy **responsiveness** (one thread keeps working while another blocks — a UI thread stays live while a worker loads data), **easy resource sharing**, **economy** (creating and switching threads is far cheaper than processes — see the context-switch discussion under *Processes*), and **scalability** across cores. The costs are the concurrency hazards: **data races**, **deadlocks**, **oversubscription** (more runnable threads than cores → context-switch thrash), and **false sharing** (independent variables sharing a cache line ping-ponging between cores).

4.3 User-level versus kernel-level threads

Threads can be managed by a user-space library the kernel never sees, or by the kernel directly.

	User-level threads	Kernel-level threads
Managed by	A user library	The OS kernel
Kernel awareness	Sees only the one process	Schedules each thread individually
Switch cost	Very cheap (no mode switch)	Higher (mode switch)
A blocking system call	Blocks the whole process (all its threads)	Blocks only that thread
Multicore	Cannot use multiple cores	Runs threads in parallel on many cores

Trap. The fatal flaw of pure user-level threads is the **blocking-syscall problem**: the kernel sees one process, so when any thread makes a blocking call, the kernel blocks the entire process and every other thread stalls with it.

4.4 Multithreading models

These describe how user threads map onto kernel threads.

Model	Mapping	Strengths	Weaknesses	Seen in
Many-to-one	Many user $\rightarrow$ 1 kernel thread	Very cheap switches	A blocking call freezes all; no true multicore	Early Java green threads (obsolete)
One-to-one	Each user $\rightarrow$ its own kernel thread	True parallelism; a block affects only one thread	Thread creation is a kernel operation; OS caps the count	Linux (NPTL), Windows, macOS — today's default
Many-to-many	Many user $\leftrightarrow$ a smaller pool of kernel threads	Parallelism <i>and</i> bounded kernel overhead	Complex two-level scheduling	Go runtime, thread pools
Two-level	Many-to-many, but a thread may be <i>bound</i> 1:1	Flexibility plus dedicated execution for critical threads	Most complex	Older Solaris, HP-UX

4.5 Concurrency versus parallelism

Definition. **Concurrency** is *dealing with* many tasks at once — a matter of structure; interleaving tasks on one core by time-slicing is concurrent. **Parallelism** is *executing* many tasks at the same instant — it requires multiple cores.

One chef switching between dishes is concurrent; several chefs cooking simultaneously is parallel. A single core can be fully concurrent (time-slicing) but never truly parallel.

Amdahl's law bounds the payoff of parallelism: if a fraction P of the work is parallelizable across N processors,

$$\text{Speedup} = \frac{1}{(1 - P) + P/N}.$$

Interview. *Max speedup of a program that is 20% strictly sequential, on 1000 cores? As $N \rightarrow \infty$ the speedup approaches $1/(1 - P) = 1/0.2 = 5 \times$. The serial fraction, not the core count, is the ceiling — the central lesson of Amdahl's law.*

4.6 Thread pools

Spawning a fresh thread per task is expensive and, if requests are unbounded, can exhaust the system. A **thread pool** creates a fixed set of worker threads up front; they wait on a shared work queue, and each picks up a task, runs it, and returns to wait. This **bounds concurrency** (no thrashing from too many threads) and removes per-task creation latency. It is the many-to-many idea applied at the application level.

4.7 Thread safety, reentrancy, and thread-local storage

- **Thread-safe** — code behaves correctly when run by several threads at once (achieved by locks, atomics, or holding no shared mutable state).
- **Reentrant** — code that can be safely interrupted mid-execution and entered again before the first call finishes; it holds no static/global mutable state and returns no pointers to static data. **All reentrant code is thread-safe, but not all thread-safe code is reentrant** — a function that guards itself with a mutex is thread-safe, yet if a signal handler re-enters it, it self-deadlocks on that mutex.
- **Thread-local storage (TLS)** — data that looks global but has a private instance per thread (C's `errno`, Java's `ThreadLocal<T>`).

Interview. *What happens if a thread in a multithreaded process calls `fork`?* The child gets **only the calling thread**, not the others. Any mutex the vanished threads held stays locked forever in the child — which is why after `fork` a child should generally only `exec` or call async-signal-safe functions.

4.8 Lightweight threads: fibers, green threads, coroutines

Modern runtimes add threads *below* the kernel's radar, scheduled cooperatively in user space so that millions can exist cheaply:

- **Go goroutines** — multiplexed many-to-many onto a small pool of OS threads; they yield at I/O and channel operations.
- **Java virtual threads (Project Loom)** — yield implicitly on blocking I/O, mapping vast numbers of virtual threads onto a few carrier OS threads.
- **Coroutines** — cooperatively scheduled, yielding at explicit `await` / `yield` points.

These trade preemption for scale: enormous concurrency at the cost of relying on tasks to yield.

5 CPU scheduling

5.1 Scheduling fundamentals

When several processes sit in the **ready queue**, the CPU scheduler selects one to run. The choice is governed by two things: the **criterion** the algorithm keys on (arrival time, burst length, priority, or a time quantum) and its **mode**:

- **Non-preemptive** — once a process holds the CPU it runs until it finishes its burst (or voluntarily blocks). A newly arrived, more deserving process must wait.
- **Preemptive** — the running process can be interrupted and returned to the ready queue if another process becomes more deserving under the algorithm's criterion.

5.1.1 Timing attributes

Each process is described by its **Arrival Time (AT)** — when it enters the ready queue — and its **Burst Time (BT)** — the CPU time it requires. The scheduler's behaviour is then evaluated through four derived quantities.

Definition — Completion Time (CT). The instant at which a process finishes execution and leaves the system.

Definition — Turnaround Time (TAT). Total time a process spends in the system, from arrival to completion:

$$TAT = CT - AT$$

Definition — Waiting Time (WT). Time spent waiting in the ready queue, i.e. turnaround minus the time actually spent executing:

$$WT = TAT - BT$$

Definition — Response Time (RT). Time from arrival until the process is *first* given the CPU — not its completion:

$$RT = (\text{time of first CPU allocation}) - AT$$

In a non-preemptive algorithm a process runs in one unbroken stretch, so its first CPU allocation coincides with the start of its only execution interval; there $RT = WT$. The two diverge only under preemption, where a process may begin, be interrupted, and resume later.

5.1.2 The algorithms at a glance

Algorithm	Criterion	Mode
First Come First Serve (FCFS)	Arrival time	Non-preemptive
Shortest Job First (SJF)	Burst time	Non-preemptive
Shortest Remaining Time First (SRTF)	Burst time (remaining)	Preemptive
Round Robin (RR)	Time quantum	Preemptive
Priority scheduling	Priority	Preemptive (or non-preemptive)
Multilevel Queue	Per-queue priority	—
Multilevel Feedback Queue	Per-queue, with movement	—

5.2 First Come First Serve (FCFS)

FCFS allocates the CPU in order of arrival: the process that requested the CPU earliest runs first. The criterion is **arrival time** and the mode is **non-preemptive**.

Worked example.

Process	AT	BT	CT	TAT	WT	RT
P1	0	2	2	2	0	0
P2	1	2	4	3	1	1
P3	5	3	8	3	0	0
P4	6	4	12	6	2	2

P1 arrives first and runs over $[0, 2]$. P2 (arrived at 1) follows over $[2, 4]$. After P2 the CPU is **idle** from 4 to 5, because the next process, P3, does not arrive until time 5; this is a key feature of FCFS — the CPU can sit idle even with work pending in the future. P3 then runs $[5, 8]$ and P4 runs $[8, 12]$.

Gantt chart

Time	0–2	2–4	4–5	5–8	8–12
CPU	P1	P2	idle	P3	P4

The derived values follow directly from $TAT = CT - AT$ and $WT = TAT - BT$. For P4: $TAT = 12 - 6 = 6$ and $WT = 6 - 4 = 2$.

Trap. FCFS is simple but suffers the *convoy effect*: a long burst arriving early forces many short jobs to wait behind it, inflating average waiting time.

5.3 Shortest Job First (SJF) — non-preemptive

Among the processes that have **already arrived**, SJF selects the one with the smallest **burst time**. The criterion is burst time; the mode here is **non-preemptive**, so once a job starts it runs to completion.

Worked example.

Process	AT	BT	CT	TAT	WT	RT
P1	1	3	6	5	2	2
P2	2	4	10	8	4	4
P3	1	2	3	2	0	0
P4	4	4	14	10	6	6

The CPU is idle over $[0, 1]$ until the first arrivals at time 1 (P1 and P3). Of those two, P3 has the shorter burst (2 vs 3), so P3 runs first $[1, 3]$. At time 3 the arrived set is {P1, P2}; P1's burst (3) is shorter, so P1 runs $[3, 6]$. Next P2 runs $[6, 10]$, and finally P4 runs $[10, 14]$.

Gantt chart

Time	0–1	1–3	3–6	6–10	10–14
CPU	idle	P3	P1	P2	P4

Averages

$$\text{Average TAT} = \frac{5 + 8 + 2 + 10}{4} = \frac{25}{4} = 6.25$$

$$\text{Average WT} = \frac{2 + 4 + 0 + 6}{4} = \frac{12}{4} = 3$$

Trap — ties. When two processes have equal burst time (here P2 and P4 both have 4), break the tie by arrival order (and, failing that, by process number). P2 arrived earlier, so it is scheduled before P4.

Key point. Non-preemptive SJF gives the minimum average waiting time for a *fixed* set of jobs available together, but it cannot react to a short job arriving while a long job is already running — that limitation motivates the preemptive variant below.

5.4 Shortest Remaining Time First (SRTF) — preemptive SJF

SRTF is the preemptive form of SJF. At every instant the CPU runs the process with the smallest **remaining** burst time; the arrival of a process with a shorter remaining time **preempts** the current one. Criterion: burst (remaining) time. Mode: preemptive.

Worked example.

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	9	9	4	0
P2	1	3	4	3	0	0
P3	2	4	13	11	7	7
P4	4	1	5	1	0	0

Tracing the remaining times:

- $t = 0$: only P1 is present; it runs. (P1 remaining $5 \rightarrow 4$ by $t = 1$.)
- $t = 1$: P2 arrives with burst 3, which is less than P1's remaining 4, so P2 preempts P1 and runs.
- $t = 2$: P3 arrives (burst 4); the running P2 has remaining 2, still the smallest, so P2 continues.
- $t = 4$: P2 completes (CT 4). P4 has just arrived with burst 1 — the smallest — so P4 runs.
- $t = 5$: P4 completes (CT 5). The contenders are P1 (remaining 4) and P3 (remaining 4); the tie is resolved in P1's favour, and P1 runs to completion at $t = 9$.
- $t = 9$: P3 runs last, completing at $t = 13$.

Gantt chart

Time	0–1	1–4	4–5	5–9	9–13
CPU	P1	P2	P4	P1	P3

Note how response time and waiting time now differ. P1 starts immediately ($RT = 0$) but is preempted, so its $WT = 4$. P3 is never preempted but waits a long time before its first slice: $RT = 9 - 2 = 7$, equal to its waiting time here.

5.5 Round Robin (RR)

Round Robin shares the CPU fairly by giving each ready process a fixed **time quantum (TQ)**. A process runs for at most one quantum, then — if unfinished — is preempted and sent to the **back** of the ready queue, while the next process is dispatched. The criterion is the time quantum; the mode is preemptive. Conceptually each process cycles between the **Ready** and **Running** states.

Worked example ($TQ = 2$).

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	12	12	7	0
P2	1	4	11	10	6	1
P3	2	2	6	4	2	2
P4	4	1	9	5	4	4

The ready-queue discipline is the crux. When a process's quantum expires, every process that arrived *during* that quantum is enqueued **before** the preempted process is re-added.

- $[0, 2]$ P1 runs (remaining $5 \rightarrow 3$). During this slice P2 ($t=1$) and P3 ($t=2$) arrive.
- $[2, 4]$ P2 runs (remaining $4 \rightarrow 2$). P4 arrives at $t = 4$.
- $[4, 6]$ P3 runs and finishes (CT 6).
- $[6, 8]$ P1 runs (remaining $3 \rightarrow 1$).
- $[8, 9]$ P4 runs and finishes (CT 9).
- $[9, 11]$ P2 runs and finishes (CT 11).
- $[11, 12]$ P1 runs its last unit and finishes (CT 12).

The resulting **sequence of processes in the ready queue** is therefore P1, P2, P3, P1, P4, P2, P1.

Gantt chart

Time	0–2	2–4	4–6	6–8	8–9	9–11	11–12
CPU	P1	P2	P3	P1	P4	P2	P1

Key point. Each boundary between two different processes in the Gantt chart is a **context switch**. RR guarantees a small response time (no process waits longer than $(n - 1) \times TQ$ for its first slice) at the cost of more context switches.

Trap. Response time is *not* the same as waiting time here. P3, for instance, first gets the CPU at $t = 4$, so $RT = 4 - 2 = 2$, while its waiting time also happens to be 2; but P1 has $RT = 0$ and $WT = 7$ because it is repeatedly preempted.

5.6 Priority scheduling (preemptive)

Each process carries a **priority** value; the scheduler always runs the highest-priority ready process. In the preemptive form, a newly arrived higher-priority process immediately displaces the running one.

Convention (this example). *Higher number = higher priority.* (The numeric convention is arbitrary and must be stated for each problem — the I/O example that follows uses the opposite convention.)

Worked example — priorities 10, 20, 30, 40 with larger meaning more urgent.

Priority	Process	AT	BT	CT	TAT	WT
10	P1	0	5	12	12	7
20	P2	1	4	8	7	3
30	P3	2	2	4	2	0
40	P4	4	1	5	1	0

- $[0, 1]$ P1 runs (only process present).
- $t = 1$: P2 (priority 20 > 10) preempts P1 and runs $[1, 2]$.
- $t = 2$: P3 (priority 30) preempts P2 and runs $[2, 4]$ to completion (CT 4).
- $t = 4$: P4 (priority 40, highest) runs $[4, 5]$ to completion (CT 5).
- $[5, 8]$ the remaining highest priority is P2 (20 > P1's 10); it finishes (CT 8).
- $[8, 12]$ P1 finally completes (CT 12).

Gantt chart

Time	0–1	1–2	2–4	4–5	5–8	8–12
CPU	P1	P2	P3	P4	P2	P1

Trap — starvation. A continuous stream of high-priority arrivals can keep a low-priority process (here P1) waiting indefinitely. The standard remedy is **aging**: gradually raising the priority of a process the longer it waits.

5.7 Priority scheduling with I/O bursts

Real processes alternate between CPU activity and I/O. This example models each process as a **CPU burst** → **I/O burst** → **CPU burst** sequence and schedules the CPU bursts by priority (preemptive). While a process is doing I/O it is not contending for the CPU, which frees the CPU for others and changes the schedule substantially.

Convention (this example). *Lower number = higher priority*, so priority 1 is the highest and 4 is the lowest — the reverse of the previous example.

Process	AT	Priority	CPU	I/O	CPU
P1	0	2	1	5	3
P2	2	3	3	3	1
P3	3	1 (<i>highest</i>)	2	3	1
P4	3	4 (<i>lowest</i>)	2	4	1

Resulting completion times: P1 = 10, P2 = 15, P3 = 9, P4 = 18.

Working through the timeline (priority 1 beats 2 beats 3 beats 4, preemptive on CPU bursts):

- [0, 1] P1 runs its first CPU burst, then leaves for I/O (returns at $t = 6$).
- [1, 2] CPU **idle** – nothing has arrived yet (P2, P3, P4 arrive at 2, 3, 3).
- [2, 3] P2 starts (it is alone).
- $t = 3$: P3 (priority 1) arrives and preempts P2; P3 runs [3, 5], finishing its first CPU burst, then leaves for I/O (returns at $t = 8$).
- [5, 6] P2 resumes (P4 is lower priority).
- $t = 6$: P1 returns from I/O (priority 2) and preempts P2; P1 runs [6, 8].
- $t = 8$: P3 returns from I/O (priority 1, highest) and preempts P1; P3 runs [8, 9] – its final CPU burst – and **completes at 9**.
- [9, 10] P1 resumes and **completes at 10**.
- [10, 11] P2 runs; this ends its first CPU burst, so it leaves for I/O (returns at $t = 14$).
- [11, 13] P4 runs its first CPU burst, then leaves for I/O (returns at $t = 17$).
- [13, 14] CPU idle (everyone is in I/O).
- [14, 15] P2 returns and runs its final CPU burst – **completes at 15**.
- [15, 17] CPU idle (P4 still in I/O).
- [17, 18] P4 returns and runs its final CPU burst – **completes at 18**.

Gantt chart

Time	0– 1	1– 2	2– 3	3– 5	5– 6	6– 8	8– 9	9– 10	10– 11	11– 13	13– 14	14– 15	15– 17	17– 18
CPU	P1	idle	P2	P3	P2	P1	P3	P1	P2	P4	idle	P2	idle	P4

Concurrent I/O intervals (overlapping the CPU timeline above):

Process	In I/O during
P1	1 $\rightarrow$ 6
P3	5 $\rightarrow$ 8
P2	11 $\rightarrow$ 14
P4	13 $\rightarrow$ 17

Key point. Overlapping I/O with CPU work is exactly why a process can “leave” and “return,” producing idle CPU gaps (1–2, 13–14, 15–17) even while jobs are technically in the system. The schedule cannot be read off burst lengths alone — the I/O return times drive it.

5.8 Multilevel Queue (MLQ) scheduling

Instead of one ready queue, MLQ partitions processes into **several separate queues by type**, each with its own scheduling algorithm and a **fixed priority**. A process is permanently assigned to one queue and never moves between them.

In the configuration shown, three queues feed the CPU:

Priority	Queue (process type)	Scheduling algorithm
Highest	System processes	Round Robin
Medium	Interactive processes	Shortest Job First
Lowest	Batch processes	First Come First Serve

A higher-priority queue is generally served entirely before a lower one is considered. This contrasts with ordinary scheduling, where a single ready queue ($P_1, P_2, \dots, P_n$) feeds the CPU directly, as drawn on the right of the figure.

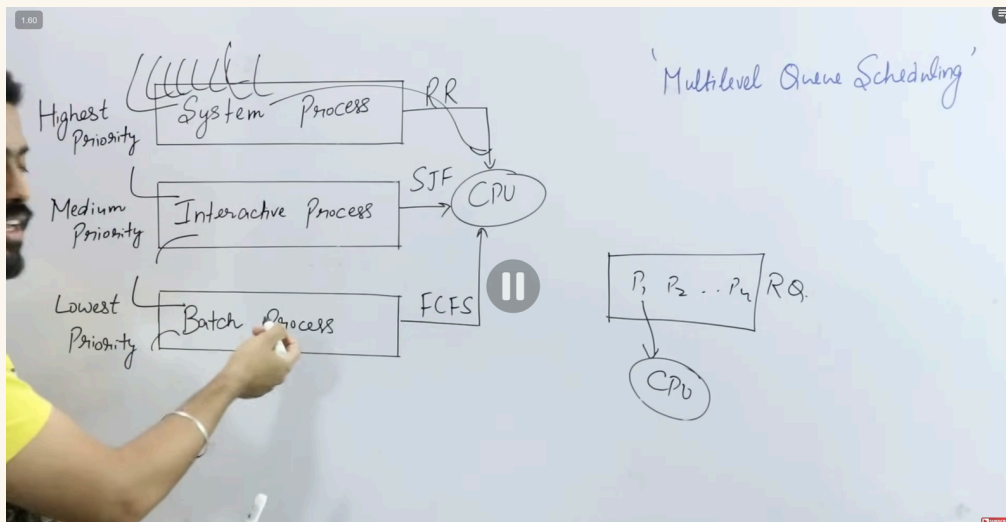


Figure 1: Multilevel Queue scheduling: three fixed-priority queues (system / interactive / batch), each with its own algorithm, contrasted with a single ready queue feeding the CPU.

Key point. The defining property of MLQ is **no migration** — a batch process can never be promoted into the system-process queue. This rigidity is what the Multilevel Feedback Queue relaxes.

5.9 Multilevel Feedback Queue (MLFQ) scheduling

MLFQ keeps multiple queues but allows a process to **move between them** based on its behaviour. A process that uses up its time quantum without finishing is **demoted** to the next queue, which grants a larger quantum. The final queue runs FCFS, absorbing the long-running jobs.

The configuration shown uses four queues, with the time quantum doubling at each level:

Queue	Discipline	Priority (as labelled in source)
RQ ₁	Time quantum = 2	Lowest (<i>batch</i>)
RQ ₂	Time quantum = 4	—
RQ ₃	Time quantum = 8	—
RQ ₄	FCFS	Highest (<i>system</i>)

A process enters RQ₁. The rule at every level above the last is the same: **if it completes, it leaves the system; otherwise it falls through to the next queue**. Whatever remains by RQ₄ is “executed in FCFS manner.”

Worked trace. A process with a total burst of 19 is whittled down as it descends:

$$19 \xrightarrow{\text{RQ}_1, \text{run } 2} 17 \xrightarrow{\text{RQ}_2, \text{run } 4} 13 \xrightarrow{\text{RQ}_3, \text{run } 8} 5 \xrightarrow{\text{RQ}_4, \text{FCFS, run } 5} 0$$

The four service slices sum to $2 + 4 + 8 + 5 = 19$, exhausting the burst.

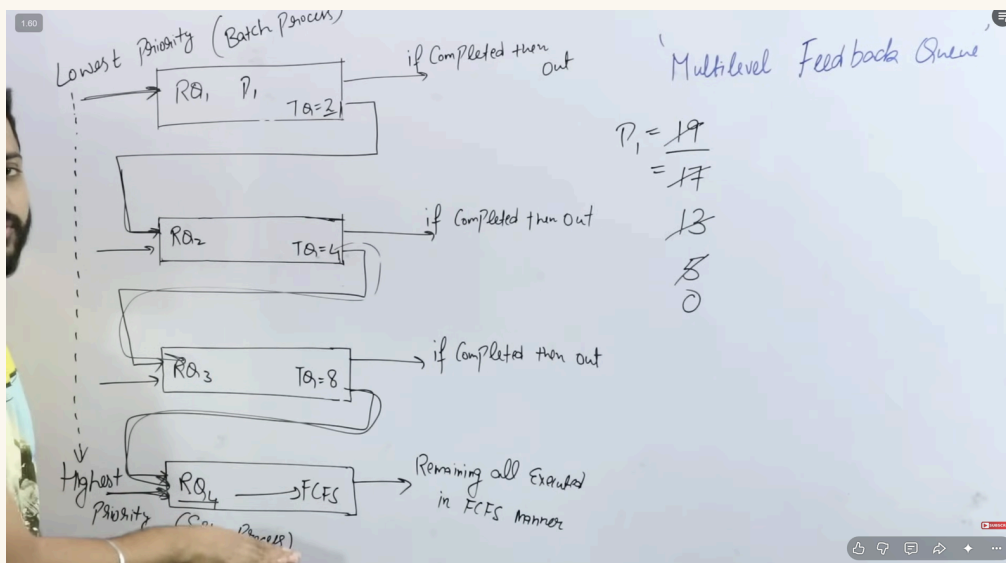


Figure 2: Multilevel Feedback Queue: a process flows RQ1 (TQ 2) → RQ2 (TQ 4) → RQ3 (TQ 8) → RQ4 (FCFS), being demoted each time it fails to finish within its quantum; the worked trace reduces a burst of 19 to 0.

Key point. MLFQ approximates “shortest job first” *without* knowing burst times in advance: short jobs finish in the early, small-quantum queues and leave quickly, while long jobs sink to the FCFS queue. The doubling quanta (2, 4, 8) limit how often a long-running job is preempted.

5.10 Assessment templates: average-turnaround drills

Online assessments (Cisco’s among them) reuse two scheduling templates almost verbatim: “given arrival and burst times, find the average turnaround time under preemptive SJF” and “...under preemptive priority.” Both are pure simulations — build the Gantt chart, decide at **every arrival and every completion**, then average $TAT = CT - AT$. The two fully worked examples below are the templates to drill.

5.10.1 Template A — Preemptive SJF (SRTF), five processes

Process	AT	BT	CT	TAT	WT
P1	0	8	18	18	10
P2	2	4	7	5	1
P3	3	1	4	1	0
P4	5	3	12	7	4
P5	6	2	9	3	1

Deciding at each event by smallest **remaining** time: P1 runs alone from 0; at $t = 2$ P2 (burst 4) undercuts P1's remaining 6 and preempts it; at $t = 3$ P3 (burst 1) preempts P2; P3 finishes at 4, after which P2 (remaining 3) is shortest and runs to completion at 7; then P5 (2), P4 (3), and finally the long P1 tail.

Gantt chart

Time	0–2	2–3	3–4	4–7	7–9	9–12	12–18
CPU	P1	P2	P3	P2	P5	P4	P1

$$\text{Average TAT} = \frac{18 + 5 + 1 + 7 + 3}{5} = \frac{34}{5} = 6.8$$

Trap. The single most common error is forgetting to preempt when a shorter job arrives mid-run (here $P2 \rightarrow P3$ at $t = 3$). Re-evaluate the ready set at **every** arrival, not just at completions. Note also that SRTF starves the long job: P1 arrived first yet finishes last.

5.10.2 Template B — Preemptive priority, five processes

Here a **smaller priority number means higher priority** (priority 1 is the most urgent). This convention is the reverse of the earlier preemptive-priority example, and flipping it is the classic misread on these questions — always confirm which way the problem defines it.

Process	AT	BT	Priority	CT	TAT
P1	0	7	4	14	14
P2	1	3	2	5	4
P3	2	1	1 (<i>highest</i>)	3	1
P4	4	3	3	8	4
P5	6	2	5 (<i>lowest</i>)	16	10

At $t = 1$ P2 (priority 2) preempts P1 (priority 4); at $t = 2$ P3 (priority 1) preempts P2 and finishes at 3; P2 then resumes and completes at 5; P4 (priority 3) runs next while P1 (4) and the not-yet-more -urgent arrivals wait; P1 finally runs its long remainder, and the lowest-priority P5 runs last.

Gantt chart

Time	0–1	1–2	2–3	3–5	5–8	8–14	14–16
CPU	P1	P2	P3	P2	P4	P1	P5

$$\text{Average TAT} = \frac{14 + 4 + 1 + 4 + 10}{5} = \frac{33}{5} = 6.6$$

Trap — starvation and convention. Because urgency, not burst length, drives the choice, a low-priority process (P5) is pushed to the very end. And the *only* thing separating a correct answer from a wrong one is often the priority convention: read “lower number = higher priority” versus “higher number = higher priority” before touching the Gantt chart.

6 Process synchronization

6.1 Independent and cooperative processes

Processes fall into two categories by how they interact:

- **Independent processes** neither affect nor are affected by other processes; they share no state.
- **Cooperative processes** do interact, because they **share** something. Synchronization is concerned exclusively with this second category.

What cooperative processes can share:

Shared item	Examples
Variables	a common counter or flag
Memory	a shared memory region
Code	re-entrant routines used by several processes
Resources	CPU, printer, scanner

Key point. Sharing is the prerequisite for a synchronization problem. If processes are truly independent, their interleaving is irrelevant; the moment they share writable state, the *order* of their operations starts to matter.

6.2 The race condition

A **race condition** arises when the final result of shared data depends on the precise interleaving of operations from several processes — an order the programmer does not control.

Setup. A shared variable is initialized on a uniprocessor:

```
int shared = 5;
```

Two cooperative processes each read it, modify a private copy, sleep, then write back:


```
// Process P1
int x = shared; // x = 5
x = x + 1;      // x = 6
sleep(1);
shared = x;     // shared = 6
```

```
// Process P2
int y = shared; // y = 5
y = y - 1;      // y = 4
sleep(1);
shared = y;     // shared = 4
```

The **expected result** is that one increment and one decrement cancel, leaving `shared == 5`. But with the interleaving above, **both** processes read `shared` as 5 *before* either writes back (the `sleep` makes this overlap concrete). Each then overwrites the variable from its own stale copy:

- if P1 writes last, `shared` ends at 6;
- if P2 writes last, `shared` ends at 4.

The outcome is non-deterministic — it “races” — and is wrong in both cases.

Definition — Race condition. A situation in which two or more processes access shared data concurrently and the final value depends on the timing/order of their execution. The root cause here is that read–modify–write is not performed as one indivisible (atomic) step; preventing it requires controlling access to the shared section of code.

6.3 The producer–consumer (bounded-buffer) problem

The canonical illustration of a race is the **bounded buffer**. A producer deposits items into a shared buffer of `n` slots; a consumer removes them. A shared counter `count` tracks how many slots are filled, and `in / out` index the next slot to fill/empty.

```
// shared
int count = 0;          // filled slots
item buffer[n];        // n = 8 in the worked example
int in = 0, out = 0;

// Producer
while (true) {
    produce_item(itemP);
    while (count == n); // buffer full — busy wait
    buffer[in] = itemP;
    in = (in + 1) % n;
    count = count + 1;
```

```

}

// Consumer
while (true) {
    while (count == 0); // buffer empty — busy wait
    itemC = buffer[out];
    out = (out + 1) % n;
    count = count - 1;
    consume_item(itemC);
}

```

The bug hides in `count = count + 1` and `count = count - 1`. Each is really three machine operations — **load** `count` into a register, **increment/decrement** the register, **store** it back:

Producer: load Rp, count	Consumer: load Rc, count
incr Rp	decr Rc
store count, Rp	store count, Rc

If the two interleave — producer loads (say `count = 3`), consumer loads the same 3, each modifies its own register, and they store in turn — one update is lost: `count` ends at 2 or 4 instead of 3. The read–modify–write on `count` is not atomic.

Key point. The producer’s and consumer’s updates to `count` form a critical section. The whole apparatus of the following sections — locks, test-and-set, semaphores — exists to make that update indivisible.

6.4 The printer–spooler problem

The same non-atomic-index bug appears in printing. A **spooler directory** `SD` holds queued files, and a shared variable `in` marks the next free slot. Each process that wants to print does:

```

1. load Ri, in          // read the shared next-free index
2. store SD[Ri], "file-name" // place this file in that slot
3. incr Ri
4. store in, Ri         // write the index back

```

If two processes both read `in` as 3, both write their file into slot 3 (the second overwrites the first) and both store `in = 4`. One print job silently vanishes even though `in` advanced only once.

Trap. These problems look different (a counter, a print queue) but are the *same* defect: an unprotected read–modify–write on a shared index. Recognizing that shared-state signature is the first step in any concurrency question.

6.5 The critical-section problem

Definition — Critical section. The part of a program where shared resources are accessed. The goal of synchronization is to ensure that when one process is executing in its critical section, no other process is allowed to execute in its own.

Each process is structured as **entry section** → **critical section** → **exit section** → **remainder**:

```
while (true) {  
    /* entry section */ // acquire permission  
    /* critical section */ // touch shared state  
    /* exit section */ // release permission  
    /* remainder */ // non-critical work  
}
```

A correct solution must satisfy four requirements. The first two are the **primary** requirements; the last two are **secondary**:

1. **Mutual exclusion** (*primary*) — at most one process in the critical section at a time.
2. **Progress** (*primary*) — if no process is in the critical section, a process that wants to enter must not be blocked by one that has no interest in entering; the decision cannot be postponed indefinitely.
3. **Bounded waiting** (*secondary*) — there is a limit on how many times other processes can enter ahead of a waiting process, so no one is starved.
4. **No assumption about hardware** (*secondary*) — correctness must not depend on the relative speed of processors or the number of CPUs.

6.6 Software and hardware solutions

6.6.1 Lock variable

The most obvious idea — a shared `LOCK` that is 1 when the critical section is busy — does *not* work:

```
1. while (LOCK == 1); // entry: spin until free  
2. LOCK = 1;  
3. // critical section  
4. LOCK = 0; // exit
```

Two processes can both execute step 1 while `LOCK` is still 0, both find it free, and both set it to 1 and enter. The test (step 1) and the set (step 2) are separate, non-atomic operations, so the lock variable **fails mutual exclusion**.

6.6.2 Test-and-set instruction

Hardware fixes this by making test-and-set a single atomic instruction: it reads the old value and writes `TRUE` in one indivisible step.

```
boolean test_and_set(boolean *target) {
    boolean r = *target;
    *target = TRUE;
    return r;
}

// entry: spin until test_and_set returns FALSE (lock was free)
while (test_and_set(&lock));
// critical section
lock = FALSE; // exit
```

Because the read and write happen atomically, only one process can observe `lock` as `FALSE` and enter — this **guarantees mutual exclusion**. (Compare-and-swap is the more general modern form of the same idea.)

6.6.3 Turn variable (strict alternation)

A pure-software two-process attempt uses a shared `turn` :

```
int turn = 0;
// Process P0      // Process P1
while (turn != 0);  while (turn != 1);
// critical section // critical section
turn = 1;           turn = 0;
```

This provides mutual exclusion, but it **violates progress**: the two processes must strictly alternate. If P0 finishes and sets `turn = 1`, P0 cannot re-enter until P1 has taken its turn — even if P1 is busy in its remainder and does not want the critical section. A process is blocked by one that has no interest in entering.

Key point — Peterson’s solution. The correct two-process software solution combines the turn variable with a `flag[]` array of intent: a process sets `flag[self] = true`, sets `turn` to the *other* process, and waits only while the other both wants in and it is the other’s turn. This satisfies all four requirements and is the standard software answer, though in practice hardware primitives and semaphores are used.

6.7 Semaphores

Definition — Semaphore. An integer variable, accessed only through two atomic operations **down** (also *P*, or *wait*) and **up** (also *V*, or *signal*), used by concurrent cooperative processes to achieve synchronization. A process that cannot proceed is put to sleep on a **suspended list** rather than busy-waiting.

6.7.1 Counting semaphore

A counting semaphore ranges over all integers and is used to guard a pool of identical resources:

```
down(S):          up(S):
  S.value = S.value - 1;    S.value = S.value + 1;
  if (S.value < 0) {        if (S.value <= 0) {
    // block this process    // select a process from the
    // add to suspended list // suspended list
    sleep();                wakeup();
  }                        }
```

A negative `S.value` counts exactly how many processes are asleep waiting on the semaphore.

6.7.2 Binary semaphore

A binary semaphore holds only 0 or 1 and is the natural mutual-exclusion lock:

```
down(S):          up(S):
  if (S.value == 1)  if (suspended list is empty)
    S.value = 0;      S.value = 1;
  else {             else {
    // block this process // select a process from the
    // add to suspended list // suspended list
    sleep();          wakeup();
  }                  }
```

Trap — semaphore vs mutex. A semaphore is a *signalling* device with no notion of an owner: any process may *up* it. A mutex is a *lock* with ownership — only the locker may unlock. A binary semaphore can serve as a mutex, but a mutex cannot provide a semaphore's cross-process signalling.

6.7.3 A worked question: maximum processes in the critical section

Suppose `n` processes each execute the same code with a semaphore `mutex` initialized to 1:

```
down(mutex);
// critical section
up(mutex);
```

How many processes can be inside the critical section at once? With `mutex = 1`, exactly the first `down` succeeds (setting the value to 0); every other process blocks. So **at most one** process is in the critical section — mutual exclusion holds. If `mutex` were instead initialized to `k` (a counting semaphore), up to `k` processes could be inside simultaneously.

6.8 Classic synchronization problems

6.8.1 Producer–consumer with semaphores

Semaphores make the bounded buffer correct. Two counting semaphores track slots and one binary semaphore `S` guards the buffer:

```
empty = n; // number of empty slots
full = 0; // number of filled slots
S = 1; // binary semaphore (mutual exclusion)

// Producer          // Consumer
produce_item(itemP);   down(full);
down(empty);          down(S);
down(S);              itemC = buffer[out];
    buffer[in] = itemP;    out = (out + 1) % n;
    in = (in + 1) % n;     up(S);
up(S);                up(empty);
up(full);              consume_item(itemC);
```

`down(empty)` blocks the producer when the buffer is full; `down(full)` blocks the consumer when it is empty; `S` ensures only one of them touches the buffer at a time.

Trap — ordering matters. The mutex `down(S)` must come *after* the counting semaphore (`down(empty)` / `down(full)`), never before. If a process took `S` first and then blocked on a full/empty buffer, it would sleep while holding the mutex — and the process that could wake it can never acquire `S`. That is a classic **deadlock**.

6.8.2 Readers–writers

Many readers may share a database concurrently, but a writer needs exclusive access. The conflict table: **read–write, write–read, write–write are problems; read–read is not**. A reader count `rc` is guarded by `mutex`, and `db` locks the database:

```

int rc = 0;
mutex = 1; // protects rc
db = 1; // database lock

// Reader
while (true) {
    down(mutex);
    rc = rc + 1;
    if (rc == 1) down(db);
    up(mutex);
    // read the database
    down(mutex);
    rc = rc - 1;
    if (rc == 0) up(db);
    up(mutex);
}

// Writer
while (true) {
    down(db);
    // write to DB
    up(db);
}

```

The **first** reader to arrive locks `db` (shutting out writers); the **last** reader to leave releases it. Writers simply take `db` exclusively.

Trap — writer starvation. This “readers-preference” version lets a continuous stream of readers keep `db` locked forever, starving writers. Fair variants give waiting writers priority.

6.8.3 Dining philosophers

Five philosophers sit around a table with five forks between them; each needs its **left and right fork** to eat. Model each fork as a semaphore `S[i]` initialized to 1:

```

S[i] = 1 for i in 0..4; // five fork semaphores

void philosopher(int i) {
    while (true) {
        think();
        wait(S[i]);           // take left fork
        wait(S[(i+1) % N]);   // take right fork
        eat();
        signal(S[i]);         // put down left fork
        signal(S[(i+1) % N]); // put down right fork
    }
}

```

Trap — deadlock. If every philosopher picks up their **left** fork at the same instant, all five forks are held and each waits forever for a right fork that its neighbour holds — a circular wait, i.e. deadlock. The standard fixes: (1) allow **at most four** philosophers at the table at once; (2) an **asymmetric** rule — odd philosophers take left-then-right, even take right-then-left; or (3) pick up both forks only if **both** are free (guarded by a mutex).

6.9 The concurrency toolkit in practice

The lecture problems are solved with semaphores; production code and coding interviews use a broader vocabulary built on the same ideas.

Locks — mutex vs spinlock. A **mutex** puts a blocked waiter to *sleep* (no wasted CPU, but a context-switch/wakeup latency of roughly a microsecond or more). A **spinlock** *busy-waits* on an atomic flag — near-zero acquire latency but every spinning cycle is burned. Spin only when the critical section is shorter than a context switch (a rule of thumb of a few microseconds) or in a context that cannot sleep (an interrupt handler); otherwise use a mutex. In user space a spinlock is dangerous: if the lock holder is preempted, every spinner wastes a full time slice.

Monitors and condition variables. A **monitor** bundles shared data with a mutex and one or more **condition variables**, guaranteeing only one thread is active inside at a time. A condition variable lets a thread sleep until a predicate holds:

```
lock(m);
while (!predicate) cond.wait(m); // always a loop, never a bare if
// ... use the shared state ...
unlock(m);
```

Trap. Always re-check the predicate in a `while` loop, never an `if`. A thread can wake **spuriously** (with no matching signal) or find the condition already consumed by another thread. `wait` atomically releases the mutex and re-acquires it on wake, closing the lost-wakeup race.

Atomics, compare-and-swap, and the ABA problem. Lock-free code relies on atomic read-modify-write hardware instructions, chiefly **compare-and-swap**: `CAS(addr, expected, new)` sets `*addr = new` only if `*addr == expected`. Because CAS checks *value*, not *history*, it is vulnerable to the **ABA problem** — a location changes $A \rightarrow B \rightarrow A$, so a stale CAS succeeds as if nothing happened. Fixes tag the pointer with a version counter, use load-linked/store-conditional, or defer memory reclamation (hazard pointers).

Memory ordering. CPUs and compilers reorder memory operations, so one thread's writes can become visible to another out of order. A **memory barrier** forbids reorderings; the acquire/release pairing (write data, then `release`-store a flag; `acquire`-load the flag, then read the data) is

how a lock or semaphore also *publishes* memory. Note that `volatile` in C/C++ prevents compiler caching but provides **no** atomicity or cross-thread ordering — use `std::atomic` instead.

False sharing. Two threads updating independent variables that happen to share one 64-byte cache line force the line to bounce between cores on every write, silently serializing them. Pad or align hot per-thread data to its own cache line.

Deadlock, livelock, starvation in code. Deadlock in code is almost always a **circular lock order** (T1 holds A wants B; T2 holds B wants A). The first-line defence is a **global lock ordering** — always acquire locks in the same fixed order. **Livelock** is threads endlessly reacting to each other without progress (fix: randomized backoff); **starvation** is a thread never getting the resource (fix: fairness/aging); **priority inversion** — a low-priority thread holding a lock a high-priority thread needs — is solved by **priority inheritance**.

7 Deadlocks

7.1 What a deadlock is

Definition — Deadlock. A situation in which a set of processes is permanently blocked, each waiting for a resource that another process in the set holds. Because the awaited event (a resource release) can never happen, none of them can ever proceed.

A deadlock requires **four conditions to hold simultaneously** (the Coffman conditions). All four are necessary; breaking any one makes deadlock impossible.

1. **Mutual exclusion** — at least one resource is held in a non-shareable mode; only one process can use it at a time.
2. **Hold and wait** — a process holds at least one resource while waiting to acquire others.
3. **No preemption** — a resource cannot be forcibly taken from a process; it must be released voluntarily.
4. **Circular wait** — there is a cycle of processes $P_0, P_1, \dots, P_n, P_0$ in which each waits for a resource held by the next.

Trap. Deadlock, starvation, and livelock are different. *Deadlock* is a permanent circular block. *Starvation* is a process that could run being indefinitely denied a resource (e.g. a low-priority job under a stream of high-priority ones). *Livelock* is processes actively changing state in response to each other yet making no progress.

7.2 Resource-allocation graphs

A **resource-allocation graph (RAG)** models the system as a directed graph. Processes are drawn as circles and resource types as boxes (with a dot per instance). A **request edge** runs from a process to a resource; an **assignment edge** runs from a resource instance to a process.

7.2.1 Single-instance resources

When every resource type has exactly one instance, the graph tells the whole story:

Key point. With single-instance resources, **a cycle in the RAG is both necessary and sufficient for deadlock** — a cycle means deadlock, no cycle means no deadlock.

For example, if P_1 holds R_1 and requests R_2 , while P_2 holds R_2 and requests R_1 , the two request/assignment edges form a cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$: a deadlock.

7.2.2 Multiple-instance resources

When a resource type has several instances, a cycle is only a *warning*:

Key point. With multiple-instance resources, **a cycle is necessary but not sufficient**. A cycle *may* be a deadlock, or another process holding an instance of a resource in the cycle may release it and break the wait. To decide, you must run the detection/safety algorithm on the allocation and request matrices — the graph alone is inconclusive.

7.3 Handling deadlocks

There are four strategies, from doing nothing to full detection:

- **Deadlock ignorance (the Ostrich algorithm)** — pretend deadlocks never happen; if the system hangs, reboot. Because deadlocks are rare, this is what most general-purpose operating systems (Linux, Windows) actually do — the cost of prevention is not worth paying continuously.
- **Deadlock prevention** — design the system so that one of the four necessary conditions can never hold.
- **Deadlock avoidance** — allow the conditions but never grant a request that could lead to deadlock, using advance knowledge of maximum needs (the Banker's algorithm).
- **Deadlock detection and recovery** — let deadlocks happen, detect them, then recover.

7.4 Deadlock prevention

Prevention attacks the four conditions directly:

Condition	How to break it	Cost
Mutual exclusion	Make resources shareable (e.g. read-only, or spooling) where possible	Not all resources can be shared
Hold and wait	Require a process to request <i>all</i> resources at once, up front	Low utilization; starvation
No preemption	Allow the OS to preempt a resource (take it back)	Only works for saveable state (CPU, memory), not printers
Circular wait	Impose a global ordering on resource types; a process may only request resources in increasing order	The most practical rule — used in real lock-ordering disciplines

Key point. Breaking **circular wait** with a global resource ordering is the technique that carries straight into code: always acquire locks in the same fixed order and a cyclic wait — and thus deadlock — becomes impossible.

7.5 Deadlock avoidance: the Banker's algorithm

Avoidance keeps the system in a **safe state**: one for which there exists an ordering of all processes — a **safe sequence** — such that each can obtain its maximum needs, run, and release, using only the resources currently available plus those freed by earlier processes in the sequence. A request is granted only if the resulting state is still safe. An **unsafe** state is not necessarily deadlocked, but it *may* lead to deadlock, so avoidance refuses to enter it.

Each process declares its **maximum** claim in advance. The **remaining need** is $\text{Need} = \text{Max} - \text{Allocation}$.

Worked example. Three resource types with totals $A = 10, B = 5, C = 7$:

Process	Allocation (A B C)	Max (A B C)	Need = Max - Alloc (A B C)
P1	0 1 0	7 5 3	7 4 3
P2	2 0 0	3 2 2	1 2 2
P3	3 0 2	9 0 2	6 0 0
P4	2 1 1	4 2 2	2 1 1
P5	0 0 2	5 3 3	5 3 1

Total allocated is $\langle 7, 2, 5 \rangle$, so the currently **Available** vector is $\langle 10, 5, 7 \rangle - \langle 7, 2, 5 \rangle = \langle 3, 3, 2 \rangle$.

The safety algorithm. Repeatedly find a process whose $\text{Need} \leq \text{Available}$; “run” it (it can finish), then return its allocation to Available . If all processes can be finished this way, the state is safe.

Step	Process run ($\text{Need} \leq \text{Available}?$)	Available after it releases
start	—	3 3 2
1	P2 (1 2 2 $\leq$ 3 3 2 ✓)	3 3 2 + 2 0 0 = 5 3 2
2	P4 (2 1 1 $\leq$ 5 3 2 ✓)	5 3 2 + 2 1 1 = 7 4 3
3	P5 (5 3 1 $\leq$ 7 4 3 ✓)	7 4 3 + 0 0 2 = 7 4 5
4	P1 (7 4 3 $\leq$ 7 4 5 ✓)	7 4 5 + 0 1 0 = 7 5 5
5	P3 (6 0 0 $\leq$ 7 5 5 ✓)	7 5 5 + 3 0 2 = 10 5 7

Every process finished, so the state is **safe**, with **safe sequence** $\langle P_2, P_4, P_5, P_1, P_3 \rangle$. (Other safe sequences may also exist.)

7.5.1 Checking whether a given ordering is safe

Assessments often ask the inverse: “*which of these orderings is **not** a safe sequence?*” The method is the same simulation, applied to the given order — walk it left to right, and at each process check $\text{Need} \leq \text{current Available}$; if it holds, add that process’s allocation back and continue. The ordering is **unsafe the moment some process’s Need exceeds the Available** at its turn.

In the example above, any sequence that **starts with P1** is not safe: P1’s need $\langle 7, 4, 3 \rangle$ already exceeds the initial Available $\langle 3, 3, 2 \rangle$, so P1 cannot begin, and nothing has yet been released to help it.

Trap. “Is X a safe sequence?” is a **simulation**, not a formula — run the Available vector forward through the ordering. If a fresh request is granted first (as in the Cisco template where a process’s request is granted before testing), subtract it from Available and add it to that process’s allocation *before* you start walking.

7.6 Detection and recovery

If a system neither prevents nor avoids deadlock, it can run a **detection** algorithm (essentially the safety algorithm using current requests instead of maximum needs) periodically or when utilization drops. On finding a deadlock it **recovers** by either:

- **Process termination** — abort all deadlocked processes, or abort them one at a time until the cycle breaks (choosing victims by priority, progress, or resources held); or
- **Resource preemption** — take a resource from a process and give it to another, rolling the victim back to a safe checkpoint (raising the risk of starvation if the same process is always chosen).

7.7 How many resources guarantee no deadlock

A classic exam template asks for the resource count that makes deadlock impossible. Suppose there are n processes $P_1, \dots, P_n$ competing for a single resource type, where P_i needs a maximum of d_i units. The worst case just short of everyone finishing is each process holding one less than it needs, i.e. $d_i - 1$ units — a total of $\sum (d_i - 1)$ units all tied up. One more unit then lets some process reach its maximum, finish, and release. Hence:

$$\text{Deadlock is possible} \Leftrightarrow R \leq \sum_{i=1}^n d_i - n, \quad \text{Deadlock-free} \Leftrightarrow R \geq \sum_{i=1}^n (d_i - 1) + 1 = \sum_{i=1}^n d_i - n + 1.$$

When every process needs the same maximum k , this reduces to the well-known form $R \geq n(k - 1) + 1$.

Worked checks.

- *Three processes, each needing 2 units — minimum units of R so no deadlock can occur?* Here $n = 3$, $d_i = 2$: deadlock-free needs $R \geq 3(2 - 1) + 1 = 4$. So **4** units suffice (with 3, each could hold 1 and wait forever).
 - *A system has 4 instances shared by 3 processes; the largest maximum request k that always avoids deadlock?* Solve $4 \geq 3(k - 1) + 1 \Rightarrow k - 1 \leq 1 \Rightarrow k \leq 2$. So the largest safe value is **$k = 2$** .
-

8 Memory management

8.1 The goal and the memory hierarchy

Memory management is how the operating system manages primary (main) memory so that many processes can reside in it at once. The CPU reads instructions and data only from registers and, below them, from cache and main memory; the disk (secondary memory) is far slower. The aim is **efficient utilization of main memory** — keeping the CPU busy by holding enough processes in RAM.

Definition — Degree of multiprogramming. The number of processes held in main memory at once. Raising it improves CPU utilization (there is more likely to be a ready process when one blocks), up to the point where memory pressure causes thrashing (covered under virtual memory).

The techniques divide into two families:

- **Contiguous allocation** — each process occupies one continuous block of memory. Either **fixed (static) partitioning** or **variable (dynamic) partitioning**.

- **Non-contiguous allocation** — a process is scattered across memory in pieces: **paging**, **multi-level/inverted paging**, **segmentation**, and **segmented paging**.

8.2 Fixed (static) partitioning

Main memory is divided in advance into a **fixed number of partitions of fixed size**; each holds at most one process, and a process must fit entirely within a single partition.

Trap — Cisco-style multi-select. All of the following are true of fixed partitioning: the number of partitions is fixed; each partition's size is fixed at configuration time; a process occupies exactly one partition and cannot span partitions; and it suffers **both** internal **and** external fragmentation. The “both fragmentations” fact is the option people wrongly deselect.

Its four limitations:

1. **Internal fragmentation** — a process smaller than its partition wastes the leftover space *inside* it (a 6 MB process in an 8 MB partition wastes 2 MB).
2. **Limit on process size** — a process larger than the biggest partition cannot run at all.
3. **Limit on the degree of multiprogramming** — at most one process per partition, so the number of partitions caps how many processes can be resident.
4. **External fragmentation** — a free partition may be unusable because no waiting process is small enough for it, so free memory exists but cannot be used.

8.3 Variable (dynamic) partitioning

Here partitions are created **to fit each process exactly**, at load time. This removes internal fragmentation and the fixed limits on process size and process count. But as processes come and go, memory becomes pocked with **holes** — **external fragmentation**: the total free memory may be large, yet split into pieces none of which is big enough for the next process.

Two remedies:

- **Compaction** — periodically slide the allocated blocks together to merge the holes into one big free region. It works but is expensive (it must pause and relocate processes).
- **Non-contiguous allocation** (paging) — the more powerful fix, which sidesteps external fragmentation entirely.

8.3.1 Placement algorithms

When several holes can fit a process, which do you pick?

Algorithm	Rule	Trade-off
First fit	The first hole large enough	Fast; good general choice
Next fit	Like first fit, but resume searching from the last placement	Spreads usage; can be worse than first fit
Best fit	The smallest hole that fits	Wastes least <i>now</i> , but leaves many tiny useless holes; slow (scans all)
Worst fit	The largest hole	Leaves a usable remainder, but consumes big holes; slow

8.4 Paging

Paging removes external fragmentation by decoupling a process's logical layout from its physical placement.

8.4.1 Pages, frames, and the page table

Physical memory is divided into fixed-size **frames**; logical memory into **pages** of the *same size*. A process's pages are loaded into any free frames, not necessarily contiguous. A per-process **page table** maps each page number to the frame that holds it. The hardware **memory-management unit (MMU)** performs this translation on every access.

Key point. Because page size equals frame size, there is **no external fragmentation** (any free frame fits any page). Only the last page of a process may be partly empty — a bounded **internal fragmentation** of at most one page per process.

8.4.2 Address translation

A logical address splits into a **page number** and a **page offset**; the page number indexes the page table to get a **frame number**, which is concatenated with the (unchanged) offset to form the physical address.

$$\text{logical address} = (\text{page number} \mid \text{offset}) \xrightarrow{\text{page table}} (\text{frame number} \mid \text{offset}) = \text{physical address}$$

Worked example (memory byte-addressable):

- Logical address space = 4 GB = 2^{32} B, so a **logical address is 32 bits**.
- Physical address space = 64 MB = 2^{26} B, so a **physical address is 26 bits**.
- Page size = frame size = 4 KB = 2^{12} B, so the **offset is 12 bits**.

From these:

Quantity	Computation	Result
Number of pages	$2^{32}/2^{12}$	2^{20}
Number of frames	$2^{26}/2^{12}$	2^{14}
Page-number bits / frame-number bits	$32 - 12 / 26 - 12$	$20 / 14$
Number of page-table entries	one per page	2^{20}
Size of the page table	entries $\times$ frame-number bits $= 2^{20} \times 14$ bits	≈ 2.5 MB

Trap. A page-table entry stores the **frame number** (14 bits here), not the full physical address — the offset is not stored. Multiply the number of entries by the frame-number width, not by the page size.

8.4.3 Page-table entry format

Each entry holds the frame number plus control bits:

- **Valid/invalid (present/absent)** — whether the page is currently in a frame.
- **Protection** — read/write/execute permissions.
- **Reference** — set when the page is accessed (used by replacement algorithms).
- **Dirty (modified)** — set when the page is written, so a clean page need not be written back.
- **Caching** — whether the page may be cached.

8.4.4 Multilevel paging

A single page table can itself be huge (2^{20} entries above). **Multilevel (hierarchical) paging** pages the page table: an **outer page table** points to **pages of the inner page table**, which point to frames. Only the portions of the table actually needed are kept in memory, so the table no longer has to be one large contiguous block. The logical address gains an index per level.

8.4.5 Inverted page table

A normal page table has one entry per *page*, per process — costly when there are many large address spaces. An **inverted page table** has one entry per *frame* of physical memory (one table for the whole system), each recording which `<process id, page number>` currently occupies that frame. This shrinks the table dramatically but makes lookup slower — the entry must be *searched* for rather than indexed — so a hash table (and the TLB) is used to speed it up.

8.5 Segmentation

Paging is invisible to the programmer and ignores a program's logical structure. **Segmentation** instead divides a process into **variable-size, logically meaningful segments** — code, stack,

heap, a data array — each identified by a segment number. A **segment table** stores, per segment, a **base address** and a **limit** (length).

Translation of a logical address (segment number, offset) : look up the segment's base and limit; if $\text{offset} < \text{limit}$, the physical address is $\text{base} + \text{offset}$; otherwise the access is illegal and the hardware raises a **trap** (segmentation fault).

Key point — paging vs segmentation. Paging uses fixed-size pages, is invisible to the programmer, and suffers only minor internal fragmentation; segmentation uses variable-size logical segments, is visible and matches program structure, but suffers **external** fragmentation. **Segmented paging** combines them — segments divided into pages — to get logical structure without external fragmentation.

9 Virtual memory

9.1 Demand paging

Definition — Virtual memory. A technique that lets a process execute even when it is not entirely in physical memory, by keeping only the actively used pages in RAM and the rest on a disk **backing store (swap)**. It gives each process the illusion of a large, contiguous address space larger than physical memory.

Virtual memory is realized by **demand paging**: a page is brought into a frame only when it is first referenced, never in advance. Each page-table entry carries a **valid/invalid** bit — valid means the page is in a frame, invalid means it is on disk (or illegal).

The page-fault sequence. When the CPU references a page whose entry is invalid, the hardware raises a **page fault** (a trap), and the OS:

1. checks that the reference is legal;
2. finds a free frame (or frees one via page replacement);
3. schedules a disk read to bring the page into the frame;
4. updates the page table (entry now valid);
5. restarts the instruction that faulted.

Key point. A page fault is a normal, expected event, not an error — it is the very mechanism that makes demand paging work. But because it involves a disk access, it is enormously slower than a memory reference, so the *rate* of page faults dominates performance.

9.1.1 Effective memory access time

If the page-fault rate is p (with $0 \leq p \leq 1$), memory access time is m , and servicing a fault costs s (dominated by disk I/O), the **effective memory access time** is:

$$\text{EMAT} = (1 - p) \cdot m + p \cdot s.$$

Because s (milliseconds) is roughly a hundred thousand times m (nanoseconds), even a tiny fault rate inflates EMAT sharply — which is why keeping p near zero, through good replacement and enough frames, is critical.

9.2 The translation lookaside buffer

Every logical reference needs a page-table lookup, and the page table itself lives in memory — so a naive design costs **two** memory accesses per reference (one for the table, one for the data). A **translation lookaside buffer (TLB)** is a small, fast associative cache of recent page→frame translations that removes most of that cost.

On a reference, the MMU checks the TLB first. A **TLB hit** yields the frame immediately; a **TLB miss** falls back to the page table (and loads the translation into the TLB). With TLB access time c , memory access time m , and TLB hit ratio h :

$$\text{EMAT} = h(c + m) + (1 - h)(c + 2m).$$

Worked example. TLB access $c = 10$ ns, memory access $m = 50$ ns, hit ratio $h = 0.8$:

$$\text{EMAT} = 0.8(10 + 50) + 0.2(10 + 50 + 50) = 0.8(60) + 0.2(110) = 48 + 22 = 70 \text{ ns}.$$

The 80 % of references that hit the TLB cost one memory access; the 20 % that miss pay for the extra page-table access.

9.3 Page replacement

When a page must be brought in but no frame is free, the OS chooses a **victim** page to evict (writing it back first only if its **dirty** bit is set). The choice of victim is made by a **page replacement algorithm**. All examples below use the reference string

$$7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 1, 2, 0 \quad (15 \text{ references})$$

with **three frames**.

9.3.1 FIFO and Belady's anomaly

FIFO evicts the page that has been in memory longest (a simple queue). Tracing the string:

Ref	7	0	1	2	0	3	0	4	2	3	0	3	1	2	0
f1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0
f2		0	0	0	0	3	3	3	2	2	2	2	1	1	1
f3			1	1	1	1	0	0	0	3	3	3	3	2	2
	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓

This gives **12 page faults and 3 hits**, a hit ratio of $3/15 = 20\%$.

Trap — Belady’s anomaly. Intuition says more frames should mean fewer faults. Under **FIFO** that can fail. For the string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, FIFO incurs **9 faults with 3 frames but 10 faults with 4 frames** — adding a frame made it *worse*. This is **Belady’s anomaly**, and it can only occur in algorithms (like FIFO) that are not “stack algorithms”; LRU and Optimal never exhibit it.

9.3.2 Optimal

Optimal (OPT/MIN) evicts the page that will not be used for the longest time in the **future**. On the reference string it achieves the fewest possible faults — **8 faults, 7 hits** (7/15). It is unrealizable in practice (it needs to see the future), but it is the yardstick every real algorithm is measured against.

9.3.3 Least recently used

LRU approximates Optimal by looking at the **past**: evict the page unused for the longest time, on the assumption that the recent past predicts the near future. On the reference string LRU gives **12 faults, 3 hits** — matching FIFO here, but in general better and, crucially, **free of Belady’s anomaly**. Exact LRU needs per-access timestamps or a stack; hardware approximates it cheaply with the **reference bit** (the “second-chance”/clock algorithm).

9.3.4 Most recently used

MRU does the opposite — evict the *most* recently used page. It is rarely a good general policy; on the reference string it is the worst of the four, **13 faults, 2 hits**. It helps only for specific access patterns (e.g. a large file scanned once, where the most-recently-used page is the least likely to be needed again).

Algorithm	Rule	Faults (3 frames)	Hits
Optimal	Evict furthest-future use	8	7
FIFO	Evict oldest-loaded	12	3
LRU	Evict least-recently-used	12	3
MRU	Evict most-recently-used	13	2

9.4 Thrashing

If the degree of multiprogramming is pushed too high, each process gets too few frames to hold its active pages, so almost every reference faults. The system then spends nearly all its time paging in and out rather than executing.

Definition — Thrashing. A condition in which a process (or the whole system) spends more time servicing page faults than doing useful work. As the degree of multiprogramming rises, CPU utilization climbs, peaks, and then **collapses sharply** once thrashing sets in.

The remedy is the **working-set model**: track the set of pages a process has referenced in a recent window (its *working set*) and give it enough frames to hold that set; admit new processes only when their working sets fit. If demand exceeds available frames, **reduce** the degree of multiprogramming (swap a process out) rather than adding more.

9.5 Related mechanisms

- **Copy-on-write** (introduced with `fork`) is a virtual-memory optimization: parent and child share physical frames marked read-only until one writes, at which point that single page is copied.
- **Memory-mapped files** (`mmap`) map a file's contents into a process's address space, so file access becomes ordinary memory access, paged in on demand and shared between processes.
- **Page cache** — the OS keeps recently used disk pages in otherwise-free RAM, so “free” memory is effectively used as a disk cache; dirty cached pages are written back lazily.

10 File systems

10.1 Files, attributes, and operations

Definition — File. A named collection of related information stored on secondary storage. It is the operating system's abstraction over the raw blocks of a disk: a user works with files and folders, and the file system maps those onto disk blocks and sectors.

A file carries **attributes** kept in its directory entry / control block: name, an **extension/type**, a unique **identifier**, its **location** on the device, **size**, **timestamps** (created/modified), **protection/permissions**, and possibly encryption or compression flags.

The OS provides **operations** on files as system calls: **create, read, write, delete, truncate, open/close**, and **reposition** (seek). Files are organized into a **directory** structure (a tree of folders), which is itself a special file mapping names to file control blocks.

10.2 File allocation methods

The core design question is *how a file's data blocks are laid out on disk*. There are three schemes.

10.2.1 Contiguous allocation

Each file occupies a set of **consecutive** blocks; the directory records the start block and length.

- **Advantages:** simple; **excellent read performance** — sequential reads need no seeks, and random access to block i is a direct computation ($\text{start} + i$).
- **Disadvantages:** **external fragmentation** (free space fragments into gaps); and a file is **hard to grow** — if the following blocks are taken, the file must be moved.

10.2.2 Linked allocation

Each block stores a pointer to the **next** block of the file; the directory holds only the first block. The blocks can be scattered anywhere.

- **Advantages:** **no external fragmentation**; a file can **grow** easily by linking another block.
- **Disadvantages:** effectively **sequential access only** — reaching block i means following i pointers, so random access is slow; the pointers consume space; and a single corrupted pointer can lose the rest of the file.

10.2.3 Indexed allocation

Each file has an **index block** holding the addresses of all its data blocks; the directory points to the index block.

- **Advantages:** supports **direct (random) access** (index the block table) and has **no external fragmentation**.
- **Disadvantages:** the index block is overhead (wasteful for tiny files), and a file larger than one index block can hold needs a **multilevel index**.

10.3 The Unix inode

Unix generalizes indexed allocation into a **multilevel index** that is efficient for both small and huge files. Each file is described by an **inode** containing its attributes plus a set of block pointers:

- several **direct** pointers (each to a data block) — enough for small files with no indirection;
- one **single-indirect** pointer (to a block of pointers to data blocks);
- one **double-indirect** pointer (to a block of pointers to single-indirect blocks);
- one **triple-indirect** pointer (one more level).

Small files use only the direct pointers, so their access is fast; large files reach further out through the indirect blocks. The number of pointers a block can hold is $\frac{\text{block size}}{\text{address size}}$.

Worked example. An inode has **8 direct** block addresses, one single-, one double-, and one triple-indirect block. Each **disk block is 128 B** and each **block address is 8 B**.

Pointers per block:

$$\frac{128 \text{ B}}{8 \text{ B}} = \frac{2^7}{2^3} = 2^4 = 16.$$

Maximum number of data blocks a file can address:

$$\underbrace{8}_{\text{direct}} + \underbrace{16}_{\text{single}} + \underbrace{16^2}_{\text{double}} + \underbrace{16^3}_{\text{triple}} = 8 + 16 + 256 + 4096 = 4376 \text{ blocks}.$$

Maximum file size:

$$4376 \times 128 \text{ B} = 560,128 \text{ B} = \mathbf{547 \text{ KB}}.$$

Key point. The triple-indirect level dominates the capacity ($16^3 = 4096$ of the 4376 blocks), yet a small file that fits in the 8 direct blocks pays no indirection cost at all. This asymmetry — cheap small files, still-reachable huge files — is why the inode design endures.

10.4 Free-space management

The file system also tracks which blocks are free, typically with a **bit vector** (one bit per block, set if free — fast to scan for a run of free blocks) or a **linked list** of free blocks. **Journaling** file systems additionally record intended changes in a log before applying them, so that an interrupted operation (a crash mid-write) can be replayed or rolled back, keeping the file system consistent.

11 I/O systems and disk scheduling

11.1 Disk architecture

A magnetic hard disk is a stack of **platters**, each with two **surfaces**. Each surface holds concentric **tracks**, and each track is divided into **sectors** (the smallest addressable unit). The set of tracks at the same radius across all platters is a **cylinder**. A read/write head per surface, mounted on a moving **arm**, accesses the data. Total capacity is

$$\text{surfaces} \times \text{tracks/surface} \times \text{sectors/track} \times \text{bytes/sector}.$$

11.2 Disk access time

Servicing an I/O request costs three components:

- **Seek time** — time for the arm to move the head to the target track. Usually the dominant, most-variable cost, and the one disk scheduling optimizes.
- **Rotational latency** — time for the target sector to rotate under the head. On average this is **half a rotation** ($\frac{1}{2} \times \frac{60}{\text{RPM}}$).

- **Transfer time** — time to actually read/write the data = $\frac{\text{data size}}{\text{transfer rate}}$.

Disk access time = seek time + rotational latency + transfer time .

11.3 Disk scheduling algorithms

Because seek time dominates, the OS reorders pending track requests to minimize total head movement. All examples below use the same request queue and starting position:

Track requests: **82, 170, 43, 140, 24, 16, 190**; the head starts at track **50**; the disk has 200 tracks (0–199).

Algorithm	Rule	Service order (from 50)	Total head movement
FCFS	Serve in arrival order	82,170,43,140,24,16,190	642
SSTF	Shortest seek time first (nearest request)	43,24,16,82,140,170,190	208
SCAN (elevator)	Sweep to one end, then reverse	82,140,170,190,→199,43,24,16	332
LOOK	Like SCAN but reverse at the last request	82,140,170,190,43,24,16	314
C-SCAN	Sweep up to the end, jump to 0, sweep up again	82,140,170,190,→199,→0,16,24,43	391
C-LOOK	Like C-SCAN but jump between the extreme requests	82,140,170,190,→16,24,43	341

A few sample computations (moving toward larger tracks):

- **SSTF**: $|50 - 43| + |43 - 24| + |24 - 16| + |16 - 82| + |82 - 140| + |140 - 170| + |170 - 190| = 7 + 19 + 8 + 66 + 58 + 30 + 20 = \mathbf{208}$.
- **SCAN**: up to the end and back, $(199 - 50) + (199 - 16) = 149 + 183 = \mathbf{332}$.
- **C-LOOK**: up to the last request, jump to the lowest, up again: $(190 - 50) + (190 - 16) + (43 - 16) = 140 + 174 + 27 = \mathbf{341}$.

Trap. SSTF and SCAN-family algorithms cut total movement versus FCFS, but SSTF can **starve** a request far from the head if closer requests keep arriving. The SCAN family avoids

starvation by sweeping steadily. The **circular** variants (C-SCAN/C-LOOK) give more *uniform* waiting time by always servicing in one direction, at the cost of the return jump. On **SSDs** — which have no heads or rotation — seek time vanishes and this scheduling largely loses its point.

11.4 How the CPU talks to devices: polling, interrupts, and DMA

There are three ways for the CPU to move data to and from a device, differing in how the CPU learns that the device is ready.

- **Programmed I/O / polling.** The CPU repeatedly reads the device’s status register in a busy loop until it is ready. A poll is cheap (nanoseconds), but the CPU **wastes cycles** whenever the device is idle.
- **Interrupt-driven I/O.** The device raises an **interrupt** when it has work; the CPU does other things (or sleeps) until then, and a handler runs per event. Each interrupt has a **fixed overhead** — saving/restoring context, dispatching the handler, and the cache/pipeline disruption it causes — on the order of a microsecond or two.
- **DMA (direct memory access).** A DMA controller transfers a whole block directly between the device and memory **without the CPU per word**; the CPU only sets up the transfer and is interrupted **once** at completion. Used for bulk transfers (disk, network), it frees the CPU from data movement (“cycle stealing” occasionally borrows the memory bus).

	Polling	Interrupt-driven	DMA
CPU cost when idle	High (busy-wait)	Low (other work)	Lowest
Cost per event	Very low	~μs of overhead	Near zero (one interrupt/block)
Best when	Events arrive back-to-back	Events are sparse	Bulk transfers

11.4.1 Which to use — and interrupt livelock

The right choice depends on the **event rate**.

- **Sparse events** (a keypress, a door sensor at ~1/minute): use **interrupts**. Polling would pin the CPU near 100 % just to catch a rare event.
- **Very high rate** (a 100 Gbps NIC at ~10 million packets/second): interrupts **melt down**. At 10^7 events/s and ~2 μs each, servicing interrupts alone would need $10^7 \times 2 \mu s = 20$ seconds of CPU **per second** — impossible. The machine spends all its time entering and leaving the handler and makes no forward progress. This is **interrupt livelock**; the fix is to switch to **polling** once packets stream in.

Key point — the adaptive hybrid (Linux NAPI). Real NIC drivers combine both: take the **first interrupt** to notice work has arrived, then **disable that interrupt and poll**, draining the device’s queue cheaply while it stays busy; **re-arm the interrupt only when a poll finds the queue empty**. The switch is driven by whether each poll still finds work — interrupt-when-idle, poll-when-busy — capturing the best of both. **Interrupt coalescing** (one interrupt per batch of packets) is a simpler hardware relative.

11.4.2 Bounded queues and tail latency

A related design choice on a latency-sensitive path is how to implement a bounded work queue:

- A **fixed-capacity ring buffer** (an array with `head / tail` indices advanced mod N) is **$O(1)$ worst-case** for both enqueue and dequeue, allocated once up front and cache-friendly, and simply **rejects** when full.
- A **two-stack queue** (an `inbox` and an `outbox`) is $O(1)$ *amortized* for dequeue — but a single dequeue that finds the outbox empty must transfer all n elements across, an **$O(n)$ worst-case spike**.

Key point. “Amortized $O(1)$ ” means the *average* over a sequence is constant — each element is handled a constant number of times (pushed to the inbox, transferred to the outbox, popped) — but it is explicitly **not** a worst-case bound. On a path with a tight, predictable deadline, choose the **ring buffer**: its $O(1)$ worst case has no tail-latency spike, whereas the two-stack queue’s periodic $O(n)$ transfer would blow the deadline even though its average is fine.

12 Protection and security

Protection controls the access of *processes and users* to the resources of a computer system — an internal mechanism. **Security** defends the system against *external and internal threats* — attackers, malware, and misuse. Protection provides the mechanisms; security sets the policy and guards the boundary.

12.1 Goals and principles

The goal of protection is to ensure that each object (memory, files, devices, CPU) is accessed **correctly, and only by processes that are allowed to** — enforcing a well-defined set of operations per object.

Key point — principle of least privilege. Every program and user should be given **just the privileges needed** to do its job, and no more. This limits the damage when a component has a bug or is compromised, and is the guiding rule behind protection design.

12.2 Domains and the access matrix

A **protection domain** is a set of **access rights**, each an `<object, {allowed operations}>` pair; a process runs within a domain and may only perform the operations its domain grants.

The **access matrix** formalizes this: **rows are domains, columns are objects**, and entry (i, j) is the set of operations a process in domain i may invoke on object j . It is a clean model of who-can-do-what; real systems implement it compactly as **access-control lists** (by column — per object, who may access it) or **capabilities** (by row — per domain, what it may touch).

12.3 Security violations

Attacks are categorized by what they compromise — the classic **CIA** triad and two more:

- **Breach of confidentiality** — unauthorized *reading* of data.
- **Breach of integrity** — unauthorized *modification* of data.
- **Breach of availability** — unauthorized *destruction* of data.
- **Theft of service** — unauthorized *use* of resources.
- **Denial of service (DoS)** — preventing legitimate *use* of the system.

Common **methods** of attack: **masquerading** (pretending to be another identity to gain privileges), **replay** (re-sending captured messages, possibly modified), **man-in-the-middle** (sitting in the data flow, impersonating each end to the other), and **session hijacking** (intercepting an already-authenticated session).

12.4 Threats

Program threats — malicious code:

- **Trojan horse** — a program that looks legitimate but hides a destructive payload; it does *not* self-replicate.
- **Virus** — a code fragment embedded in a legitimate program; it is **self-replicating** and spreads when the host program runs (often via email or shared files).
- **Worm** — a **self-replicating** program that spreads across systems **on its own**, without a host program or user action, consuming resources as it multiplies.
- **Spyware** — covertly collects information about the user; **trap door / backdoor** — a hidden entry point that bypasses normal authentication.

Trap. *Malware* is the umbrella term. The distinctions interviewers probe: a **virus** needs a host and an execution to spread; a **worm** needs neither; a **trojan** neither self-replicates nor needs a host — it relies on the user running it willingly.

System and network threats — **port scanning** (probing for open services to exploit), **denial of service** and its distributed form **DDoS** (flooding a target from many sources), and connection-flood tricks such as the **SYN** attack against the TCP handshake.

12.5 Defenses

- **Cryptography** — protect confidentiality and authenticity of messages using keys, so that only the intended recipient can read them and the sender can be verified.
 - **Firewalls** — place a barrier between trusted and untrusted networks, limiting and inspecting traffic across the boundary (packet-filter, application-proxy, and system-call firewalls).
 - **Defense in depth** — security must be enforced at **four levels** to be effective: **physical** (data centers, terminals), **human** (avoiding social engineering and phishing), **operating system** (protection mechanisms, patching), and **network** (guarding communications). Security is only as strong as the weakest link.
-

13 Appendix A — Programming paradigms

A *programming paradigm* (or methodology) is a style of structuring a program. These methodologies exist to make programming easier as systems grow; the progression below moves from everything-in-one-place toward bundling data with the operations on it. In the earliest style, both **data and instructions** live together in a single undivided program.

13.1 Monolithic

A single, basic program: one continuous block of statements with no separation into reusable units. Data and instructions are intermingled in the one program.

```
Program
{
    statement
    statement
    ...
}
```

13.2 Procedural

The program is broken into **functions** (procedures), as in the C language. Logic is organized into named routines that `main` calls in turn.

```
function1()
{
    ...
}
```

```

function2()
{
    ...
}

main()
{
    function1();
    function2();
    function3();
}

```

13.3 Modular

Related **data** is grouped into a structure, and that structure is passed to the functions that operate on it. Data and behaviour are still separate, but the data is now organized.

```

structure info {
    data1;
    data2;
    data3;
}

function1(info) { ... }
function2(info) { ... }

main()
{
    struct info i;
    function1(i);
    function2(i);
}

```

13.4 Object-oriented

Data and the functions that act on it are **bundled together** inside a `class`. Objects of the class then invoke their own member functions — the data no longer has to be passed around explicitly.

```

class info {
    data1;
    data2;
    data3;
    function1();
    function2();
};

main()

```

```
{
    info i;
    i.function1();
    i.function2();
}
```

Key point. The through-line across paradigms is increasing *cohesion* between data and behaviour: monolithic mixes everything, procedural separates behaviour into functions, modular organizes the data into structures, and object-oriented finally unites data and functions inside a single class.

14 Appendix B — Rapid-fire interview questions

A condensed set of the questions that recur in operating-systems interview rounds, each with a crisp answer. Every topic here is treated in full in the body; this appendix is for fast revision.

1. Process vs thread. A *process* is a program in execution with its own address space and resources; a *thread* is a unit of execution *inside* a process that shares the process's code, data, and heap but keeps its own stack, registers, and program counter. Threads are cheaper to create and switch, and communicate through shared memory; processes are isolated and communicate through IPC.

2. Process states and the PCB. A process moves through New → Ready → Running → Waiting/Blocked → Terminated (with Suspend-Ready and Suspend-Blocked once swapping is involved). The *process control block* is the kernel structure holding everything needed to manage and resume it: PID, state, program counter, saved registers, memory-management info, accounting, and the open-file table.

3. Mutex vs binary semaphore. A *mutex* is a locking mechanism with **ownership** — the thread that locks it must unlock it. A *binary semaphore* is a signaling mechanism with **no ownership** — one thread can wait and another can signal. A binary semaphore can act as a mutex, but a mutex cannot provide the signaling role of a semaphore.

4. Monolithic vs microkernel. A *monolithic* kernel runs all services (scheduling, memory, file systems, drivers) in one kernel address space — fast (direct calls) but a bug anywhere can crash the system. A *microkernel* keeps only the essentials in the kernel and runs drivers and file systems as user-space servers — reliable and modular, but slower because services interact through IPC and mode switches. Linux is monolithic; QNX is a microkernel.

5. What is a system call? The controlled gateway through which a user program requests a kernel service. It loads a call number and arguments into registers, executes a trap instruction to

switch to kernel mode, and the kernel dispatches through the system-call table (e.g. `fork`, `read`, `write`, `open`, `exec`). It is far costlier than an ordinary function call.

6. Demand paging. Pages are loaded into RAM only when first referenced, not all at once. A reference to a page that is not resident causes a *page fault*, and the OS brings that page in from disk. It reduces memory use and startup latency, at the cost of page-fault handling.

7. Virtual memory. An abstraction that lets a process use an address space larger than physical RAM by keeping only the actively used pages in memory and the rest on a disk backing store. It gives isolation, the illusion of a large contiguous space, and is realized through paging plus demand paging.

8. Paging vs segmentation. *Paging* divides memory into fixed-size pages/frames; it eliminates external fragmentation and is invisible to the programmer, but suffers a little internal fragmentation. *Segmentation* divides memory into variable-size, logically meaningful segments (code, stack, data) visible to the programmer; it matches program structure but suffers external fragmentation.

9. Internal vs external fragmentation. *Internal* fragmentation is space wasted *inside* an allocated block, because the block is larger than what was requested (fixed partitions, page padding). *External* fragmentation is free memory that exists but is split into scattered holes, none large enough for a request even though their sum is. Paging removes external fragmentation; compaction fixes it in contiguous schemes.

10. Preemptive vs non-preemptive scheduling. Under *non-preemptive* scheduling a running process keeps the CPU until it finishes or blocks; under *preemptive* scheduling the OS can take the CPU away (on a timer tick or the arrival of a more deserving process). Preemption improves responsiveness and fairness at the cost of more context switches.

11. Multiprogramming vs multitasking vs multiprocessing. *Multiprogramming* keeps several jobs in memory and switches the single CPU to another whenever one blocks, to maximize utilization. *Multitasking* (time-sharing) adds rapid time-slicing so many interactive tasks appear to run at once. *Multiprocessing* uses two or more physical CPUs/cores to run tasks in genuine parallel.

12. Spatial vs temporal locality. *Spatial* locality: memory near a recently accessed address is likely to be accessed soon (favouring block/line fetches). *Temporal* locality: a recently accessed location is likely to be accessed again soon (favouring caching). Caches, paging, and TLBs all exploit both.

13. Virus, worm, trojan, malware, spyware. *Malware* is the umbrella term for malicious software. A *virus* attaches to a host program and needs that program to run to spread. A *worm* is self-replicating and spreads across systems on its own, without a host or user action. A *trojan* masquerades as legitimate software and does not self-replicate. *Spyware* covertly gathers information about the user.

14. What causes a deadlock? Four conditions must hold simultaneously — *mutual exclusion*, *hold-and-wait*, *no preemption*, and *circular wait*. Breaking any one prevents deadlock; the most practical is imposing a global lock-acquisition order to remove circular wait.

15. What is thrashing? A state in which a process spends more time paging than executing, because it has too few frames for its working set, so nearly every access faults. Raising the degree of multiprogramming past a point makes CPU utilization collapse; the fix is to reduce the load or grow each process's frame allocation (working-set model).

Key point. Two more distinctions worth having ready: a **spinlock** busy-waits and suits very short critical sections or non-sleepable contexts, whereas a **mutex** sleeps the waiter and suits everything else; and a **process context switch** is expensive because it flushes the TLB, while a **thread switch** only swaps registers.